

Transmission vidéo basse latence pour drone FPV

Travail de Bachelor

Non Confidentiel

Départements : TIC

Filière : Informatique et systèmes de communication

Orientation : Systèmes informatiques embarqués

Auteur : Tschantz Nathan

Supervisé par : Favrat Pierre

Date : 15 juillet 2026

1 Préambule

Ce travail de Bachelor (ci-après **TB**) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'École.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD
Le Chef du Département

Yverdon-les-Bains, le 15 juillet 2026

2 Authentification

Je soussigné, Nathan Tschantz, atteste par la présente avoir réalisé seul ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Nathan Tschantz



Yverdon-les-Bains, le 15 juillet 2026

3 Résumé - français

Contexte : Le pilotage de drones FPV en environnements complexes nécessite un retour vidéo en temps réel. Or, les fréquences standards (2.4 et 5.8 GHz) pénètrent mal les obstacles physiques (conditions NLOS).

Problématique : La norme Wi-Fi HaLow (IEEE 802.11ah, Sub-1 GHz) offre une excellente pénétration, mais ses mécanismes natifs de fiabilité (acquittements MAC, retransmissions) induisent une latence et une gigue incompatibles avec les exigences strictes du vol en immersion.

Méthode : Ce travail implémente une architecture de bout en bout optimisée pour le temps réel. Un programme en C développé sur la Raspberry Pi isole et fragmente activement les images d'un flux vidéo MJPEG natif en paquets autonomes, transmis via SPI à un microcontrôleur STM32. La liaison radio est configurée en UDP Unicast sur une modulation fixe et robuste (MCS 1, 8 MHz), tandis que les paramètres d'accès au canal de la Qualité de Service (QoS MAC) du modem sont forcés pour éliminer les temps d'attente inter-frames.

Résultat : Après avoir isolé un temps de vol radio pur de l'ordre de 12.5 ms, l'approche itérative a permis d'éliminer les lourds goulots d'étranglement applicatifs. La latence visuelle globale absolue (*Glass-to-Glass*) du système final s'établit entre **40 et 50 ms en moyenne**. Ces performances dépassent l'objectif d'acceptabilité de 80 ms et valident la viabilité du Wi-Fi HaLow pour le pilotage en milieu confiné.

4 Abstract - English

Context : FPV drone piloting in complex environments requires a real-time video feed. However, standard frequencies (2.4 and 5.8 GHz) struggle to penetrate physical obstacles (NLOS conditions).

Problem : The Wi-Fi HaLow standard (IEEE 802.11ah, Sub-1 GHz) offers excellent obstacle penetration, but its native reliability mechanisms (MAC acknowledgments, retransmissions) introduce latency and jitter incompatible with the strict requirements of immersive flight.

Method : This work implements an end-to-end architecture tailored for absolute real-time transmission. A custom C program running on the Raspberry Pi actively isolates and fragments native MJPEG video frames into autonomous packets, which are transmitted over SPI to a STM32 microcontroller. The radio link is configured using UDP Unicast on a robust, fixed modulation scheme (MCS 1, 8 MHz), while the modem's MAC Quality of Service (QoS) channel access parameters are forced to eliminate inter-frame waiting times.

Result : After isolating a pure radio transport time of approximately 12.5 ms, the iterative approach successfully eliminated heavy application-level bottlenecks. The absolute overall *Glass-to-Glass* visual latency of the final system ranges between **40 and 50 ms on average**. These performances surpass the initial 80 ms target and definitively validate the viability of Wi-Fi HaLow for drone piloting in confined environments.

Table des matières

1	Préambule	3
2	Authentification	5
3	Résumé - français	7
4	Abstract - English	7
5	Terminologie et concepts de base	14
	Terminologie et concepts clés	14
6	Introduction	16
6.1	Contexte	16
6.2	Problématique	16
6.3	Objectifs du travail de Bachelor	16
6.3.1	Définition du seuil d'acceptabilité et cas d'usage	17
6.4	Structure du document	17
7	Étude de faisabilité	18
7.1	But	18
7.2	Méthodologie d'investigation	18
7.3	Faisabilité côté Émetteur (FreeRTOS / STM32)	18
7.4	Faisabilité côté Récepteur (Linux / OpenWrt)	18
7.5	Conclusion de la faisabilité	19
8	Architecture globale du système	20
8.1	Le module d'acquisition et d'émission embarqué (TX)	20
8.2	Le pont réseau de réception au sol (routeur RX)	21
8.3	La station de décodage et d'affichage	21
8.4	Architecture finale retenue pour le système	22
8.5	Flux logique du système	22
9	Configuration de l'environnement de développement embarqué	23
9.1	Intégration du SDK sous Windows	23
9.2	Adaptation de la chaîne de cross compilation	23
9.3	Configuration du débogage matériel (ST-LINK)	24
9.4	Provisionnement et calibration radio (Config Store)	24
9.4.1	Architecture de flashage	24
9.4.2	Résolution du conflit de signature et de version matérielle	24
10	Implémentation du réseau Wi-Fi HaLow	25
10.1	Contournement des contraintes réglementaires (ETSI vs FCC)	25
10.2	Stratégie de transport UDP : Du Broadcast à l'Unicast optimisé	26
10.2.1	L'approche initiale : UDP Broadcast et rétention DTIM	26

10.2.2	La solution finale : UDP Unicast et forçage de la QoS (WMM/EDCA)	26
10.3	Optimisation de la réactivité radio : Désactivation du PSM	27
10.4	Le routage et l'isolation IP (OpenWrt)	27
10.5	Validation de la bande passante radio : PHY vs Réelle	28
11	Capture et traitement du flux vidéo	29
11.1	La phase d'expérimentation : L'analyse des codecs inter-frames (H.264 et H.265)	29
11.1.1	Arbitrage initial entre H.264 et H.265	29
11.1.2	Limites physiques et latence plancher du H.264	29
11.2	La phase finale : Le changement de paradigme vers le MJPEG	30
11.2.1	Le prérequis indispensable : L'élargissement de la bande passante (BW)	30
11.2.2	Profilage du flux MJPEG et fragmentation réseau	30
11.3	Pipelines d'émission MJPEG final (TX)	32
11.3.1	Précisions sur les paramètres d'envoi	32
11.4	Pipeline de réception MJPEG final (RX)	33
11.4.1	Précisions sur les paramètres de réception	33
12	Interfaçage matériel et transfert SPI	34
12.1	Identification du bus SPI matériel	34
12.2	Développement de la Gateway SPI-MJPEG (C)	35
12.3	Contrôle de flux matériel (Handshake SPI)	35
12.4	Optimisation de l'ordonnancement OS et Overclocking	36
12.4.1	Overclocking et verrouillage des fréquences	36
12.4.2	Isolation du cœur CPU	36
13	Itérations matérielles et analyse des performances	37
13.1	Profilage théorique du transport matériel	37
13.1.1	Mesure au cycle d'horloge du traitement MCU (STM32)	37
13.1.2	Bilan de la latence de transport radio pur	37
13.2	Traque de la latence applicative : Le plafond de verre H.264	38
13.3	Performances et limites de l'architecture finale	38
13.3.1	Comparatif avec un système Broadcast alternatif (DVB-T2)	38
13.3.2	Validation matérielle RF et limitations de puissance	38
13.3.3	Évaluation empirique de la portée et de la pénétration (NLOS)	39
13.3.4	Qualité visuelle en conditions réelles	39
13.3.5	Bilan Glass-to-Glass final	39
14	Conclusion	41
15	Note sur l'utilisation de l'intelligence artificielle	42
16	Bibliographie	43

A	Guide de connexion et d'accès aux équipements	45
A.1	Accès au microcontrôleur d'émission (STM32 / EKH05)	45
A.2	Accès à l'ordinateur d'acquisition (Raspberry Pi 4B)	45
A.3	Accès au routeur relais de réception (GL.iNet MT3000 / OpenWrt)	45
A.4	Accès et adressage des stations de réception (RX)	46
A.5	Tableau récapitulatif d'adressage et d'accès	46
B	Modifications du pilote Morse Micro	47
C	Mesure de latence via bouton partagé	47
C.1	Le Code TX (Émetteur de test : latency_tx.c)	47
C.2	Le Code RX (Chronomètre de précision : latency_rx.c)	49
D	Code de la Gateway active finale (Raspberry Pi TX)	52
E	Firmware MCU du transmetteur (STM32U5 / spi_interface.c)	57
F	Système de transmission alternatif DVB-T2 (SDR / BladeRF)	61
F.1	Matériel et architecture logique	61
F.2	Configuration de l'émetteur BladeRF (Raspberry Pi)	62
F.3	Configuration du récepteur et décodage (PC Windows)	63
F.3.1	Approche standard : VLC Media Player	63
F.3.2	Approche optimisée : TSDuck	63
F.3.3	Pipeline d'affichage GStreamer (Commun aux deux méthodes)	64
F.4	Bilan des mesures et goulots d'étranglement	64
	Glossaire	65
	Index	67

Table des figures

1	Structure globale du système	17
2	Architecture globale du système de transmission vidéo	20
3	Architecture retenue	22
4	Flux logique du système : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo, (3) Plateformes de traitement, (4) Interfaçage SPI.	22
5	Configuration du serveur GDB ST-LINK dans STM32CubeIDE	24
6	(1) Lien réseau Sub-1 GHz	25
7	(2) Capture vidéo	32
8	(3) Plateformes de traitement.	33
9	(4) Interfaçage SPI	34
10	Schéma du système de test de latence matérielle	37
11	Captures d'écran du retour vidéo MJPEG (320x240, Qualité 8, 60 FPS) en extérieur	39
12	Architecture logique de la chaîne de transmission comparative DVB-T2 . .	62

Liste des tableaux

1	Routage du bus SPI sur la carte d'évaluation EKH05	34
2	Portée utile et comportement de la liaison en mode NLOS	39
3	Récapitulatif des accès et du plan d'adressage (Sous-réseau FPV : 192.168.12.x)	46

5 Terminologie et concepts de base

Afin de faciliter la lecture de ce document et la compréhension des enjeux techniques liés à la transmission vidéo basse latence, les concepts et acronymes suivants seront couramment utilisés :

Wi-Fi HaLow (IEEE 802.11ah) : Amendement du standard Wi-Fi conçu pour opérer dans les bandes de fréquences inférieures à 1 GHz (Sub-1 GHz). Contrairement au Wi-Fi classique (2.4 GHz ou 5.8 GHz), il privilégie la portée (jusqu'à plusieurs kilomètres) et la capacité à traverser les obstacles physiques, au détriment de la bande passante maximale.

FPV (*First Person View*) : "Vol en immersion". Technique de pilotage où la caméra embarquée sur le drone transmet un flux vidéo en temps réel vers les lunettes ou l'écran du pilote, exigeant une latence extrêmement faible et constante.

NLOS (*Non-Line-Of-Sight*) : Situation de communication radio où l'émetteur et le récepteur ne sont pas en ligne de vue directe (présence de bâtiments, de végétation ou de relief). C'est dans ces conditions que les hautes fréquences perdent leur efficacité.

MCS (*Modulation and Coding Scheme*) : Indice (généralement de 0 à 10) définissant le débit physique de la transmission. Il combine un type de modulation radio (ex : BPSK, QAM) et un taux de codage. Un MCS bas (ex : 1 ou 2) offre un débit réduit, mais garantit une robustesse maximale face au bruit et aux interférences.

LDPC (*Low-Density Parity-Check*) : Algorithme avancé de correction d'erreurs (FEC). Contrairement au codage convolutif standard (BCC), le LDPC utilise des matrices de parité complexes permettant au récepteur de reconstruire des paquets de données corrompus par de mauvaises conditions radio, évitant ainsi de devoir redemander l'envoi du paquet (ce qui détruirait la latence vidéo).

Couches PHY et MAC : Niveaux fondamentaux du modèle OSI. La **couche PHY** (Physique) gère la transformation des bits en ondes radio. La **couche MAC** (*Media Access Control*) gère les règles d'accès à l'antenne et les retransmissions. Ce projet modifie directement ces couches pour les adapter au flux vidéo.

RTOS (*Real-Time Operating System*) : Système d'exploitation temps réel (comme FreeRTOS, utilisé sur le microcontrôleur d'émission STM32). Contrairement à un OS classique (comme Linux), un RTOS garantit l'exécution des tâches dans des délais stricts et déterministes, minimisant ainsi la gigue logicielle lors du routage des données.

Latence "Glass-to-Glass" : Délai total mesuré entre l'instant où un événement physique se produit devant l'objectif de la caméra (le premier "verre") et le moment où il est affiché sur l'écran du pilote (le second "verre"). C'est la métrique de performance absolue de ce projet, englobant tous les temps de traitement matériels et logiciels.

UDP Broadcast & No-ACK : Stratégie de transmission réseau consistant à diffuser des paquets de données à l'aveugle à tous les équipements du sous-réseau, sans

demander d'accusé de réception (*No-ACK*). Cette méthode sacrifie la garantie absolue de livraison au profit d'un flux continu à latence minimale.

Gigue (*Jitter*) : Variation temporelle de la latence. Dans un flux vidéo en temps réel, une forte gigue (des paquets arrivant en rafales irrégulières) provoque des saccades visuelles souvent plus perturbantes pour le pilotage qu'une latence fixe et stable.

Provisionnement (*Provisioning*) : Processus technique consistant à injecter des données de configuration vitales, des fichiers de calibration matérielle (binaire de microcode BCF) et des profils réglementaires régionaux (*Country Code*) dans une zone mémoire non volatile dédiée (le *Config Store* du STM32). Cette opération, distincte du flashage du firmware applicatif en C, est indispensable pour permettre le démarrage d'une puce radio dépourvue de mémoire morte interne (comme le SoC Morse Micro MM8108), en lui fournissant sa feuille de route matérielle dès sa mise sous tension.

6 Introduction

6.1 Contexte

Le pilotage de drones FPV (*First Person View*) repose sur un retour vidéo en temps réel. Pour garantir la sécurité du vol et la réactivité du pilote, la fiabilité de la liaison radio et la minimisation de la latence sont absolument critiques. Actuellement, la grande majorité des retours vidéo commerciaux reposent sur des liaisons fonctionnant dans les bandes de fréquences de 2.4 GHz ou 5.8 GHz. Bien que ces hautes fréquences offrent une large bande passante permettant le transfert de vidéos en haute définition, leur capacité de pénétration est en revanche médiocre face aux obstacles physiques.

En environnement urbain, forestier ou industriel (conditions dites *Non-Line-Of-Sight* ou NLOS), la portée de ces systèmes vidéo se trouve drastiquement réduite. À l'inverse, les protocoles opérant dans les bandes inférieures à 1 GHz (Sub-1 GHz, tels que TBS Crossfire ou ExpressLRS) sont devenus le standard industriel pour le lien de contrôle (la radiocommande) grâce à leur robustesse face aux obstacles, mais leur bande passante a toujours été techniquement insuffisante pour y faire transiter de la vidéo.

6.2 Problématique

Pour pallier ce manque et réunir le meilleur des deux mondes, la norme IEEE 802.11ah, également connue sous le nom de Wi-Fi HaLow, se présente comme une alternative prometteuse. Opérant dans la bande de fréquences Sub-1 GHz, elle offre une portée kilométrique et une excellente capacité à traverser les matériaux, tout en proposant des débits de l'ordre du Mégabit par seconde.

Cependant, les standards Wi-Fi classiques n'ont pas été conçus à l'origine pour transporter des flux vidéo continus et déterministes à très faible latence. Les mécanismes natifs d'adaptation de débit (*Rate Control*) et les retransmissions automatiques de paquets visent à garantir l'intégrité absolue des données au détriment du temps de livraison. Pour le vol FPV, l'attente d'une retransmission introduit une gigue (*jitter*) et une latence totalement incompatibles avec les exigences de pilotage.

6.3 Objectifs du travail de Bachelor

L'objectif principal de ce projet est d'étudier, de concevoir et de valider de bout en bout un prototype de transmission vidéo exploitant les avantages physiques de la norme IEEE 802.11ah (Wi-Fi HaLow).

Plutôt que de se limiter à la simple configuration logicielle de modules radio, l'enjeu s'est révélé être une véritable optimisation du système dans son ensemble. Cela implique le contournement des mécanismes d'acquiescement réseau (diffusion unidirectionnelle), le réglage strict des pipelines de compression vidéo matérielle, ainsi que la mise en place d'une synchronisation physique entre les différents processeurs et microcontrôleurs embarqués pour éviter l'accumulation de données dans les mémoires tampons (*buffers*).

6.3.1 Définition du seuil d'acceptabilité et cas d'usage

Au-delà de l'optimisation technique de la chaîne, il convient de définir un objectif de latence global pragmatique, aligné avec les contraintes physiques du vol en immersion. Sur une machine de type "freestyle" ou de course, les systèmes de transmission numériques traditionnels (opérant en 5.8 GHz) ciblent une latence de 20 à 30 ms (voire inférieure) pour autoriser des manœuvres très agressives.

Toutefois, l'objectif de ce travail n'est pas de concurrencer ces systèmes sur la vitesse pure, mais de privilégier la robustesse et la pénétration de signal en milieu complexe (conditions NLOS telles que l'exploration de bâtiments isolés, de forêts denses ou de tunnels). Dans ce contexte d'exploration, une absence de perte de liaison est préférable à une latence absolue minimale.

Le pilotage reste parfaitement viable et sécurisant jusqu'à un seuil de latence d'environ 80 ms. L'objectif de ce système, une fois toutes les optimisations matérielles et logicielles appliquées, est donc d'atteindre ce seuil des 80 ms, qui représente une limite raisonnable au vu de la chaîne d'acquisition matérielle grand public exploitée dans ce projet.

6.4 Structure du document

Ce rapport s'articule autour d'une progression logique, allant de la mise en place des fondations matérielles et logicielles jusqu'à l'optimisation itérative du système complet. Après une étude de faisabilité validant nos choix technologiques, nous détaillerons l'architecture globale retenue.



FIGURE 1 – Structure globale du système

Dans un premier temps, nous aborderons la configuration de l'environnement de développement et l'implémentation spécifique du lien réseau Wi-Fi HaLow, éléments qui constituent le socle de notre système.

Une fois ce pont réseau établi, nous décrirons les mécanismes logiciels de capture et de traitement de la vidéo, puis nous détaillerons l'interfaçage matériel (via le bus SPI) permettant d'acheminer ce flux du processeur vers le microcontrôleur d'émission.

Enfin, la dernière partie de ce rapport sera consacrée à l'approche itérative du projet : où seront présentées les différentes combinaisons de plateformes matérielles évaluées (Nvidia Jetson Nano, Raspberry Pi, Debix, PC) et où sera exposée l'analyse des mesures de latence ayant permis d'isoler les différents goulots d'étranglement de la chaîne.

7 Étude de faisabilité

7.1 But

Pour réaliser une transmission vidéo robuste face aux obstacles, notre stratégie repose sur le forçage d'un MCS bas et l'utilisation de la technique de correction d'erreurs LDPC. L'objectif de cette étude de faisabilité est de déterminer si ces paramètres, généralement gérés dynamiquement par le matériel, sont configurables manuellement sur les modules Wi-Fi HaLow Morse Micro MM8108 utilisés pour ce projet.

7.2 Méthodologie d'investigation

Dans un premier temps, l'analyse de la documentation constructeur et des interfaces utilisateur (*user-friendly*) d'OpenWrt a révélé que si certains paramètres basiques sont accessibles (comme le choix du canal ou de la largeur de bande), l'activation forcée du LDPC ou le verrouillage strict du MCS ne sont pas exposés par les utilitaires standards (tels que `iw`, `morse_cli` ou `hostapd`). L'investigation s'est donc orientée vers le reverse engineering et l'analyse du code source des pilotes fournis par Morse Micro, tant pour l'environnement FreeRTOS (Émetteur embarqué) que pour l'environnement Linux (Routeur relais).

7.3 Faisabilité côté Émetteur (FreeRTOS / STM32)

L'émetteur (TX), chargé de récupérer la vidéo et d'émettre les ondes radio depuis le drone, repose sur un microcontrôleur STM32 exploitant le système d'exploitation temps réel FreeRTOS. L'investigation du kit de développement logiciel (`mm-iot-sdk`) a permis d'identifier les leviers d'action suivants :

- **Contrôle du MCS** : L'analyse des bibliothèques a mis en évidence la fonction interne `mmwlan_ate_override_rate_control()` issue du fichier d'en-tête `mmwlan.h`. Cette fonction de test (ATE) permet de verrouiller dynamiquement la bande passante et le MCS directement dans le code source de l'application en C.
- **Activation du LDPC** : L'inspection des configurations internes, notamment celles liées au service `wpa_supplicant` (`config_ssid.h`), indique que la fonctionnalité LDPC est activée par défaut par le pilote lors de la négociation de la connexion.

7.4 Faisabilité côté Récepteur (Linux / OpenWrt)

Le relais radio, configuré en point d'accès (Access Point, AP) au sol pour réceptionner le flux, repose sur une architecture Linux (OpenWrt). L'analyse du code source du pilote noyau (*kernel driver*) a révélé les éléments suivants :

- **Contrôle du MCS** : Les fichiers sources `rc.c` et `command.c` contiennent des structures permettant de désactiver l'algorithme d'adaptation de débit (`enable_fixed_rate`) et d'imposer un MCS fixe (par exemple, MCS 2). Des tests

de compilation croisée (*cross-compilation*) du noyau ont confirmé que ces modifications logicielles peuvent être compilées sous forme de module (`morse.ko`) et injectées à chaud sur le routeur cible.

- **La limite du forçage LDPC** : Initialement, une modification directe des entêtes de paquets MAC a été explorée. L'étude du fichier `skbq.c` laissait supposer la possibilité d'intercepter les trames pour manipuler le registre `morse_ratecode` (en forçant les bits B17 et B18 du champ SIG-1). Cependant, les expérimentations ont démontré qu'il est techniquement impossible d'appliquer et de maintenir ces paramètres matériels bas niveau tant que le routeur n'est pas formellement connecté à un client.

La preuve formelle d'une injection manuelle du LDPC n'a pas pu être clairement établie et a donc été écartée de l'implémentation finale. Néanmoins, la présence active des paramètres liés au LDPC et au MCS 10 dans le code source du pilote confirme la capacité de la puce à utiliser cette correction d'erreur en interne. Nous partons donc du postulat techniquement fondé que le contrôleur radio l'exploite de manière native.

7.5 Conclusion de la faisabilité

L'étude démontre que l'objectif de manipulation de la couche physique est réalisable sur les points critiques. S'il s'est avéré impraticable de forger manuellement chaque en-tête MAC pour imposer le LDPC en raison de la machine d'états du standard Wi-Fi, l'accès au code source du SDK FreeRTOS et des pilotes Linux permet d'appliquer les surcharges (*overrides*) nécessaires pour verrouiller l'utilisation d'un MCS bas. Ces découvertes valident la faisabilité d'une liaison radio déterministe, prérequis absolu pour notre système vidéo.

8 Architecture globale du système

Afin de répondre aux exigences de la transmission vidéo basse latence pour le pilotage FPV, l'architecture du système a été pensée de manière modulaire. La chaîne de transmission a été segmentée en trois entités fonctionnelles et matérielles distinctes :

1. **Le module d'acquisition et d'émission embarqué (TX)** : Destiné à être monté sur le drone, il est chargé de la capture vidéo brute, de sa compression, et de son émission radio sur la bande Sub-1 GHz.
2. **Le pont réseau de réception au sol (routeur RX)** : Il agit comme une passerelle réseau transparente. Son unique rôle est de réceptionner les ondes radio HaLow et de les convertir en trames Ethernet physiques, sans effectuer de traitement logiciel.
3. **La station de décodage et d'affichage** : Connectée au pont réseau via un câble Ethernet, cette entité est exclusivement dédiée au traitement logiciel du flux vidéo entrant pour l'afficher sur l'écran du pilote avec le délai le plus court possible.

Cette segmentation offre un double avantage : elle permet de décharger les microcontrôleurs Wi-Fi des lourdes tâches d'encodage/décodage vidéo, et elle a rendu possible le profilage de chaque sous-système séparément tout au long de la phase de développement. Voici l'ensemble des plateformes matérielles ayant été évaluées.

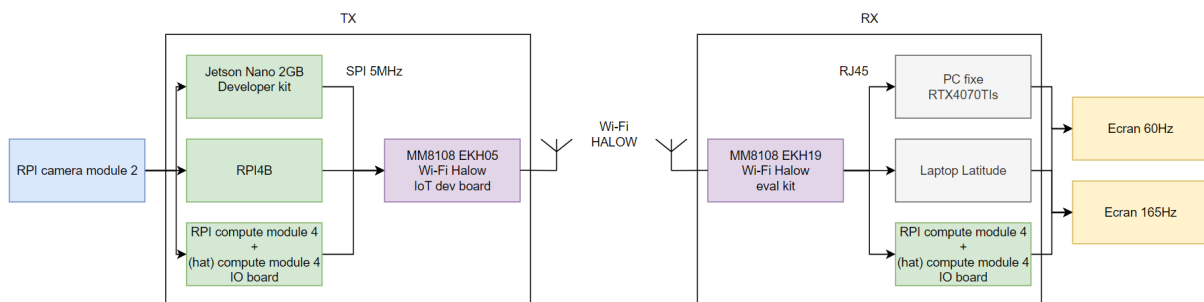


FIGURE 2 – Architecture globale du système de transmission vidéo

8.1 Le module d'acquisition et d'émission embarqué (TX)

L'émetteur embarqué constitue le point de départ de la chaîne d'acquisition. Le choix du module de traitement vidéo et d'encodage a fait l'objet d'une sélection itérative parmi plusieurs ordinateurs monocartes :

- Une **Nvidia Jetson Nano 2GB** : Évaluée lors des premières phases d'expérimentation pour valider la chaîne de transmission avec un encodeur matériel dédié (NVENC). Cette plateforme s'est avérée logiquement moins optimale pour notre application spécifique que l'écosystème Raspberry Pi. Elle a donc été écartée suite à cette première phase de validation.

- L'écosystème **Raspberry Pi 4** (regroupant la *Compute Module 4* et la *4B* classique en une seule architecture de référence) : Sélectionné comme la solution matérielle finale pour l'émission. L'accès direct et hautement optimisé au processeur d'image (ISP) via l'outil natif `rpicam-vid` s'est révélé nettement plus efficace en termes de latence brute que toutes les autres solutions testées.
- Une **Debix Model A** : Évaluée brièvement pour ses capacités théoriques. Elle a rapidement été écartée car l'exploitation de son accélération matérielle nécessitait l'utilisation de longs pipelines GStreamer qui introduisaient plus de latence logicielle que l'implémentation directe et rationalisée de `rpicam-vid` sur la Raspberry Pi.

Indépendamment de l'ordinateur monocarte retenu pour la compression, les trames segmentées sont transmises via un bus SPI cadencé à 8 MHz vers un microcontrôleur **STM32U5**. Ce MCU agit comme un pont matériel déterministe. Il encapsule les données vidéo entrantes dans des paquets UDP à l'aide de la pile réseau LwIP, avant de les expédier au module radio **Morse Micro MM8108 (EKH05)** pour la transmission RF sur la bande Sub-1 GHz.

8.2 Le pont réseau de réception au sol (routeur RX)

La réception des ondes HaLow est assurée par un routeur **GL.iNet MT3000** équipé d'une carte d'évaluation **EKH19**. Ce routeur, fonctionnant sous le système d'exploitation OpenWrt, est configuré pour agir comme un simple pont logiciel (*Bridge*) transparent. Il réceptionne les trames 802.11ah sur son interface sans-fil et les injecte directement sur son interface Ethernet physique, limitant au maximum le temps de transit et le traitement par la pile TCP/IP interne du CPU.

8.3 La station de décodage et d'affichage

La station au sol est le dernier maillon de la chaîne, responsable du décodage et de l'affichage. Deux environnements ont été employés pour observer l'impact des couches d'abstraction d'affichage :

- **La station cible (Raspberry Pi 4 / CM4)** : Utilisée lors du prototypage pour évaluer des pipelines d'affichage KMS (*Kernel Mode Setting*) afin de s'affranchir de l'overhead de l'OS.
- **La station de décodage haute performance (PC fixe, GPU NVIDIA RTX 4070 Ti)** : Bien que l'architecture cible finale d'un tel système soit embarquée, cette plateforme de diagnostic a été retenue pour l'établissement de nos mesures de performance finales. Ce choix est principalement dicté par la nécessité d'utiliser un écran haute fréquence, en l'occurrence **165 Hz**. Cette caractéristique est indispensable pour éliminer le goulot d'étranglement temporel lié au taux de rafraîchissement d'un affichage standard (60 Hz), permettant de mesurer la latence glass-to-glass au plus proche de sa valeur réelle.

8.4 Architecture finale retenue pour le système

Suite aux phases d'évaluations itératives, l'architecture optimale et stabilisée retenue pour l'évaluation finale du système s'articule autour des composants matériels et logiciels suivants :

- **Côté Émetteur (TX) :** Une caméra Raspberry Pi V2 reliée à une Raspberry Pi 4B (CPU isolé sur le cœur 3, overclocké à 2.0 GHz). Le flux vidéo est capturé et compressé à la volée au format MJPEG à l'aide de l'outil `rpikam-vid` (320x240, 60 FPS, Qualité 8). Les trames sont découpées et poussées sur le bus SPI cadencé à 8 MHz par notre passerelle en C (`gateway_spi.c`) vers la carte de transmission STM32U5/MM8108 (EKH05).
- **Liaison Radio :** Modulation fixe forcée en MCS 1 (8 MHz de largeur de bande) utilisant le protocole UDP Unicast avec un marquage agressif de la Qualité de Service MAC (WMM/EDCA file "Voice") et la désactivation totale du mode d'économie d'énergie (PSM).
- **Côté Récepteur (RX) :** La carte de réception EKH19 reliée en bus interne au routeur OpenWrt GL.iNet MT3000, configurée en simple pont Ethernet transparent vers notre PC fixe de diagnostic (décodage GStreamer logiciel sans synchronisation et affichage direct sur l'écran haute vitesse 165 Hz).

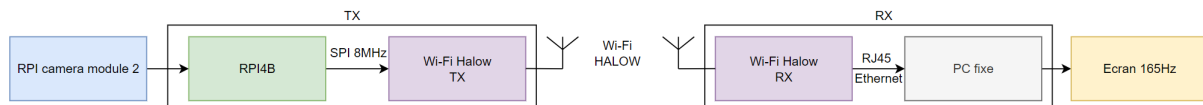


FIGURE 3 – Architecture retenue

8.5 Flux logique du système

L'articulation des composants et le cheminement des données à travers les quatre grands maillons du système sont modélisés par la figure 4.

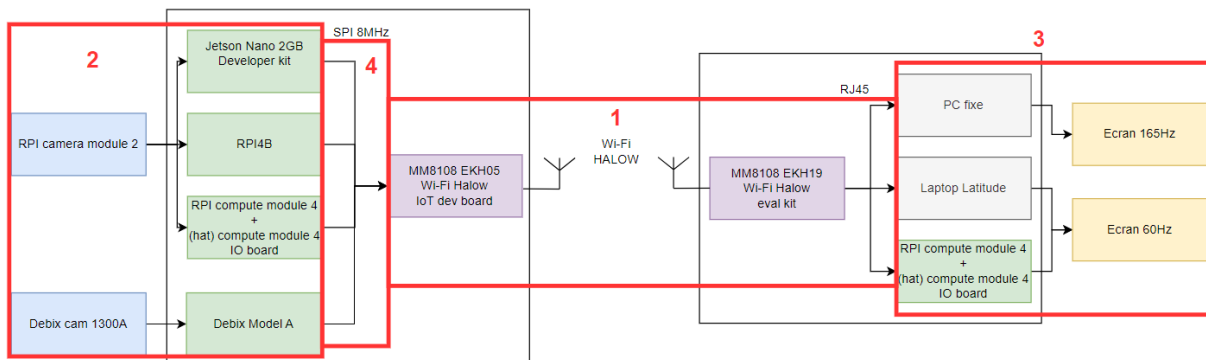


FIGURE 4 – Flux logique du système : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo, (3) Plateformes de traitement, (4) Interfaçage SPI.

9 Configuration de l'environnement de développement embarqué

Le développement du firmware pour le microcontrôleur gérant la transmission (STM32) repose sur le kit de développement logiciel officiel de Morse Micro (`mm-iot-sdk`). Ce SDK étant nativement conçu pour un environnement Linux avec Platformio et des chaînes de cross compilation préinstallées, son intégration sous Windows a nécessité plusieurs adaptations.

9.1 Intégration du SDK sous Windows

Le code source a été récupéré via le dépôt GitHub officiel. L'architecture du projet s'appuyant sur plusieurs dépendances externes vitales (telles que le système d'exploitation temps réel FreeRTOS, la pile réseau LwIP et mbedTLS), une initialisation complète des sous-modules a été requise (`git submodules`).

L'environnement de développement choisi est **STM32CubeIDE**. Le SDK ne respectant pas la structure standard de cet IDE, le projet a été importé en tant que "Makefile Project with Existing Code", en ciblant spécifiquement le répertoire de la carte d'évaluation de l'émetteur (`mm-mm6108-ekh05` avant sa mise à jour, `mm-mm8108-ekh05` après).

9.2 Adaptation de la chaîne de cross compilation

Sous Windows, les outils de compilation en ligne de commande tels que `make` ne sont pas natifs. Pour pallier ce problème, les variables d'environnement du projet dans STM32CubeIDE ont été modifiées pour inclure les chemins absolus vers les utilitaires internes de l'IDE (`make.exe` et `arm-none-eabi-gcc.exe`).

De plus, le SDK force la recherche de la chaîne de compilation GCC ARM dans les répertoires typiques de Linux (tels que `/opt/`). Il a donc été nécessaire de modifier le fichier de configuration principal `mkcore-arm-cortex-m33f.mk` pour empêcher cette recherche et permettre au système d'utiliser le `PATH` de Windows :

```
# --- MODIFICATION POUR WINDOWS / STM32CubeIDE ---
# On ignore la recherche des dossiers Linux /opt/...
# et on laisse le PATH systeme de STM32CubeIDE trouver le
#   compilateur.
TOOLCHAIN_DIR :=
TOOLCHAIN_BASE := arm-none-eabi-

CC := "$(TOOLCHAIN_BASE)gcc"
CXX := "$(TOOLCHAIN_BASE)g++"
AS := $(CC) -x assembler-with-cpp
OBJCOPY := "$(TOOLCHAIN_BASE)objcopy"
```

9.3 Configuration du débogage matériel (ST-LINK)

Pour permettre l'exécution pas à pas et la pose de breakpoint, le projet a été converti en "Projet ARM" dans l'IDE. La sonde de débogage utilisée est le **ST-LINK** intégré à la carte EKH05, communiquant via le protocole SWD (*Serial Wire Debug*).

FIGURE 5 – Configuration du serveur GDB ST-LINK dans STM32CubeIDE

Il est important de noter une limitation physique de cette architecture : la sonde USB (ST-LINK) ne peut maintenir qu'un seul tunnel de communication actif à la fois. L'utilisation simultanée de la console série (ex : PuTTY, Termit) et du débogueur GDB entraîne des conflits matériels.

9.4 Provisionnement et calibration radio (Config Store)

Une étape indispensable, distincte de la compilation du code C, est le **provisionnement** de la puce Wi-Fi. La puce Morse Micro ne possède pas de mémoire non volatile interne pour ses paramètres vitaux ; elle ne peut pas démarrer sans son micro-code (BCF) et ses paramètres régionaux (*Country Code*), stockés dans une zone mémoire spécifique du STM32 appelée le *Config Store*.

9.4.1 Architecture de flashage

Le processus de flashage repose sur une architecture client-serveur :

1. **Le Serveur (OpenOCD)** : Il établit la connexion matérielle entre le port USB et le processeur ARM via la sonde ST-LINK.
2. **Le Client (Script Python)** : Il lit le fichier de configuration utilisateur (`config.hjson`), génère le binaire correspondant, et commande au serveur de l'écrire dans la mémoire Flash du STM32 (à l'adresse `0x08004000`).

9.4.2 Résolution du conflit de signature et de version matérielle

Lors des premiers déploiements, une erreur bloquait la communication SPI entre le MCU et la puce Wi-Fi (`Transport init failed - Chip ID 0x0000`). Le code embarqué planifiait dès l'initialisation du HAL (Hardware Abstraction Layer).

L'analyse de ce blocage a révélé un désaccord matériel : le code C et le fichier de configuration cachaient une dépendance stricte à l'ancienne plateforme (MM6108). Notre matériel étant la version MM8108, le binaire de calibration injecté était incorrect. Par une coïncidence extrêmement heureuse, Morse Micro a mis à jour son SDK public la semaine suivante, ajoutant officiellement le support de la cible `mm-mm8108-ekh05`.

Cette mise à jour a permis de rediriger le script vers le fichier de calibration exact de la puce (`bcmf08651_us.mbin`). Combiné à l'ajustement du fichier `config.hjson` pour cibler la région "US", la puce radio a pu démarrer avec succès.

10 Implémentation du réseau Wi-Fi HaLow

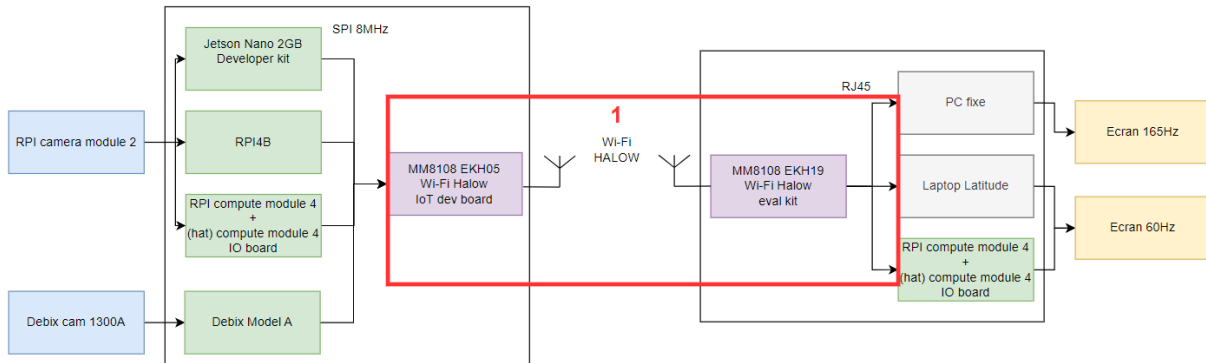


FIGURE 6 – (1) Lien réseau Sub-1 GHz

La norme IEEE 802.11 (Wi-Fi) intègre intrinsèquement des mécanismes de fiabilité, tels que les accusés de réception (ACK) matériels au niveau de la couche MAC et les retransmissions automatiques. Si ces fonctionnalités garantissent l'intégrité des données pour de l'IoT standard, elles s'opposent aux exigences du flux vidéo FPV où la perte d'une trame est largement préférable à l'attente d'une retransmission tardive. L'ensemble de la pile réseau a donc été reconfiguré pour prioriser la réactivité au détriment de la fiabilité de transport.

10.1 Contournement des contraintes réglementaires (ETSI vs FCC)

Avant d'entamer la configuration fine du flux vidéo, l'établissement de la liaison radio HaLow en Europe (norme ETSI, bande des 868 MHz) s'est heurté à un obstacle majeur : une latence de base colossale d'environ 670 ms, mesurée lors de simples paquets de test (Ping).

L'analyse de ce délai a mis en évidence l'impact restrictif des réglementations européennes régissant les bandes de fréquences Sub-1 GHz. Pour permettre la coexistence des équipements, l'ETSI impose le mécanisme *Listen Before Talk* (LBT) ainsi que des restrictions strictes de *Duty Cycle* (temps d'émission maximal autorisé cumulé). Le module radio Morse Micro bridait donc son propre débit et retenait délibérément les paquets pour respecter ces contraintes légales.

Afin de mesurer les performances physiques réelles du matériel sans ces limitations logicielles régionales, l'ensemble du système a été basculé sur le domaine réglementaire américain (FCC, bande des 915 MHz). L'utilisation du Canal 28 (916 MHz) permet de désactiver les mécanismes LBT et de s'affranchir des limites de *Duty Cycle*. Ce choix expérimental indispensable a permis de faire chuter instantanément la latence de base du ping à sa valeur matérielle brute, située entre **3 et 5 ms**, rendant le projet possible.

10.2 Stratégie de transport UDP : Du Broadcast à l'Unicast optimisé

La couche transport exploite le protocole UDP pour s'affranchir de la gestion des connexions et des retransmissions inhérentes à TCP. L'élimination des délais au niveau de la couche liaison (MAC 802.11ah) a quant à elle nécessité deux approches successives.

10.2.1 L'approche initiale : UDP Broadcast et rétention DTIM

Pour écarter la contrainte d'attente des acquittements matériels (MAC ACKs), la première stratégie consistait à émettre le flux vidéo sur l'adresse de diffusion locale (192.168.12.255). LwIP traduit cette adresse par l'adresse MAC de destination globale FF:FF:FF:FF:FF:FF. Selon la norme 802.11, les trames diffusées en *Broadcast* ne déclenchent aucun acquittement de la part des récepteurs.

Cependant, les tests d'émission ont révélé un goulot d'étranglement majeur induit par le routeur OpenWrt de réception. Selon les spécifications de la norme IEEE 802.11, les points d'accès ont l'obligation réglementaire de mettre en mémoire tampon les trames de diffusion (*Broadcast/Multicast*) et de ne les libérer qu'immédiatement après la transmission d'une balise de signalisation spécifique appelée DTIM (*Delivery Traffic Indication Message*). Ce mécanisme est conçu à l'origine pour permettre aux stations IoT de rester en veille profonde et de ne se réveiller que périodiquement pour écouter le trafic global.

Avec un intervalle de balise par défaut de 100 ms et un paramètre DTIM configuré à 2 sur OpenWrt, le point d'accès ne libérait les données accumulées que toutes les 200 ms. L'analyse des captures a mis en évidence une rétention logicielle asynchrone induisant une latence et une gigue colossales d'environ **40 ms**. Le passage en Unicast a permis de contourner immédiatement ce tampon pour envoyer chaque paquet en flux tendu.

10.2.2 La solution finale : UDP Unicast et forçage de la QoS (WMM/EDCA)

Pour contourner la rétention liée au DTIM, la liaison a été basculée en **UDP Unicast**, ciblant directement l'adresse IP unique de la station de réception (192.168.12.10). En Unicast, le point d'accès OpenWrt transmet immédiatement chaque paquet reçu sur l'interface radio.

Le retour à l'Unicast réintroduisant obligatoirement le mécanisme d'acquittement matériel (MAC ACK) sous peine de voir la puce radio couper la transmission, il a fallu configurer le modem de manière agressive au niveau de ses files d'attente de priorité d'accès au canal (EDCA/WMM).

Dans un premier temps, les paquets UDP vidéo sont marqués au niveau IP avec un drapeau TOS (*Type of Service*) prioritaire (pcb->tos = 0xC0, correspondant à la catégorie réseau *Voice*). Dans un second temps, la structure de contrôle du SDK Morse Micro a été configurée et forcée dans le firmware du STM32 :

```
// Configuration QoS ultra-agressive des files d'attente EDCA
struct mmwlan_qos_queue_params fpv_qos = {
    .aci = 3,           // Access Category Index : 3 correspond a la
                        // file d'attente "Voice"
```

```
.aifs = 2,          // Arbitration Inter-Frame Spacing : Temps d'
    attente minimal legal
.cw_min = 1,        // Contention Window Min : Taille minimale de
    la fenetre d'attente
.cw_max = 1,        // Contention Window Max : Evite l'
    augmentation du delai apres collision
.txop_max_us = 0    // Transmission Opportunity : Temps d'
    occupation maximal (0 = un seul paquet)
};
mmwlan_set_default_qos_queue_params(&fpv_qos, 1);
```

L'explication physique de ce forçage est la suivante :

- **aci = 3 (Voice)** : Oriente physiquement nos paquets vidéo vers la file d'attente matérielle la plus prioritaire du modem radio.
- **aifs = 2** : Réduit le temps d'attente obligatoire entre deux tentatives d'émission à sa limite légale absolue. Le transmetteur écoute à peine le canal avant de parler.
- **cw_min = 1 et cw_max = 1** : En Wi-Fi standard, si le canal est occupé ou qu'une collision survient, l'émetteur applique un recul aléatoire exponentiel pour retenter sa chance plus tard. En forçant ces fenêtres à leur valeur minimale (1), le STM32 retente l'émission immédiatement sans délai d'attente, écrasant virtuellement les autres trafics environnants pour imposer l'envoi de la vidéo.

L'application de ce "hack" de QoS a permis de digérer l'overhead des acquittements MAC Unicast tout en récupérant les **40 ms** perdues dans les buffers DTIM du routeur, stabilisant ainsi la liaison radio. Cela a également eu pour effet

10.3 Optimisation de la réactivité radio : Désactivation du PSM

Par défaut, le SDK Morse Micro active les profils d'économie d'énergie (*Power Saving Mode* - PSM) destinés aux capteurs autonomes de l'Internet des objets (IoT). Ce mode place temporairement le récepteur radio en veille profonde et nécessite une phase de réveil lors de chaque transaction réseau, ce qui générerait une latence systématique d'environ **22 ms** par paquet.

Le mode veille a donc été désactivé dans l'initialisation de l'application :

```
mmwlan_set_power_save_mode(MMWLAN_PS_DISABLED);
```

Ce forçage maintient la radio active à 100% de son temps, ramenant la gigue et le délai de transmission à leur minimum.

10.4 Le routage et l'isolation IP (OpenWrt)

Afin de simplifier le transit réseau, le réseau FPV est strictement isolé sur le sous-réseau 192.168.12.x.

Pour maximiser l'efficacité du routeur de réception au sol (GL.iNet MT3000 fonctionnant sous OpenWrt), la redirection des données en espace utilisateur (*User-Space*) via des utilitaires comme **netcat** ou des interceptions **tcpdump** a été proscrite en raison de la surcharge CPU qu'elle générerait. Un pont réseau (*Bridge*) a été configuré directement au niveau du noyau Linux du routeur :

```
brctl addif br-ahwlan wlan0
```

Ce pont matériel permet aux fragments vidéo UDP d'être répliqués de l'interface radio vers le port Ethernet RJ45 de manière transparente, sans aucune analyse ou rétention par les couches applicatives du routeur, économisant ainsi 10 à 20 ms de transit logiciel.

10.5 Validation de la bande passante radio : PHY vs Réelle

La désactivation des algorithmes d'adaptation automatique de débit (*Rate Control*) du modem radio a été configurée via la fonction ATE (*Automated Test Equipment*) du SDK :

```
mmwlan_ate_override_rate_control(MMWLAN_MCS_1 , MMWLAN_BW_8MHZ ,  
MMWLAN_GI_NONE);
```

En forçant l'utilisation du protocole sur le **MCS 1** avec une largeur de bande de **8 MHz**, la couche physique (PHY) affiche un débit de synchronisation théorique de ****6.4 Mbps**** (en Short Guard Interval). Il est cependant crucial de distinguer ce débit brut de la bande passante utile réellement disponible pour les données utilisateur (*data payload*).

En raison des en-têtes de protocoles empilés (PHY, MAC 802.11ah, IP, UDP), des temps d'attente inter-trames obligatoires (IFS) et surtout de la réintroduction des acquittements MAC en mode Unicast, le débit utile réel disponible au niveau applicatif s'effondre à **moins de 50 %** de la bande passante PHY théorique (soit environ 2.5 à 3 Mbps dans des conditions parfaites de laboratoire). En situation de vol réel avec un rapport signal-sur-bruit (SNR) dégradé, cette limite utile s'approche des **1.5 Mbps**. Ce constat justifie scientifiquement le dimensionnement de notre flux MJPEG, volontairement contraint et profilé sous ce seuil critique pour éviter toute saturation du canal.

11 Capture et traitement du flux vidéo

Une fois le pont réseau Sub-1 GHz configuré sur ses couches basses, le défi consiste à formater la source vidéo pour qu'elle puisse traverser ce canal sans générer de goulots d'étranglement applicatifs. Cette section détaille l'arbitrage entre les codecs, le profilage des flux et l'optimisation des pipelines.

11.1 La phase d'expérimentation : L'analyse des codecs inter-frames (H.264 et H.265)

L'étude initiale du projet s'est orientée vers les standards de compression vidéo modernes, caractérisés par un encodage temporel (*inter-frame*).

11.1.1 Arbitrage initial entre H.264 et H.265

La première architecture évaluée ciblait la norme **H.265 (HEVC)**. Ce codec offre théoriquement une efficacité de compression supérieure de 50 % par rapport au H.264, permettant de conserver une image exploitable à un débit extrêmement bas (400 kbps). Cependant, sa complexité algorithmique s'est heurtée à la réalité matérielle : l'absence d'un pilote de décodage matériel H.265 stable et performant sous Linux (`v4l2h265dec`) a provoqué une surcharge CPU massive sur nos plateformes, élevant la latence à plus de 300 ms.

Le système a donc été basculé vers la norme **H.264 (AVC)**. Ce choix a permis d'exploiter les puces de décodage matériel universelles du PC fixe de diagnostic et des Raspberry Pi, ramenant le temps de décodage en dessous des 50 ms.

11.1.2 Limites physiques et latence plancher du H.264

Malgré l'optimisation drastique des pipelines GStreamer à l'émission et à la réception (désactivation des B-Frames, profils *baseline*, forçage du mode *flush* pour vider les tampons), le système H.264 a buté sur un plafond de verre.

Les meilleures mesures de latence globale (*Glass-to-Glass*) obtenues avec ce codec se sont établies à :

- **Entre 100 ms et 120 ms** sur le PC fixe de diagnostic (bénéficiant du décodage GPU optimal et d'un affichage 165 Hz).
- **Entre 130 ms et 140 ms** sur la station de réception cible Raspberry Pi 4.

Cette latence a révélé un problème structurel : par définition, le H.264 est un codec **inter-frame**. Même configuré pour de l'ultra-basse latence, le décodeur et le parseur (`h264parse`) exigent de mettre en mémoire tampon et d'analyser les dépendances temporelles de plusieurs frames consécutives pour reconstruire l'image. Ce délai d'accumulation logiciel, couplé à l'overhead des files d'attente du système d'exploitation, n'est pas optimal pour du FPV.

11.2 La phase finale : Le changement de paradigme vers le MJPEG

Pour éliminer tout délai d'accumulation temporelle, la compression inter-frame a été abandonnée au profit du format **MJPEG** (*Motion JPEG*).

11.2.1 Le prérequis indispensable : L'élargissement de la bande passante (BW)

Le MJPEG est un codec exclusivement **intra-frame** : chaque image est compressée de manière autonome, comme une photographie JPEG isolée. Ce choix élimine instantanément le besoin de mise en mémoire tampon temporelle au décodage : une image reçue par le réseau est immédiatement affichable.

Cependant, cette absence d'analyse temporelle se traduit par un flux de données nettement plus lourd que le H.264. Lors de nos premiers essais en H.264, la bande passante radio était volontairement bridée sur une largeur de bande de **2 MHz** (MCS 2) pour maximiser la portée, ce qui limitait le débit utile à environ 400 Kbps. Faire passer un flux MJPEG dans un tel canal était physiquement impossible et causait une saturation immédiate.

La transition vers le MJPEG a donc été rendue possible uniquement par la reconfiguration matérielle de la liaison radio HaLow, basculée sur une largeur de bande de **8 MHz** (MCS 1). Ce changement de canal a permis de lever le goulot d'étranglement de la couche physique, offrant le débit nécessaire pour soutenir les exigences du format JPEG sans perte de paquets.

11.2.2 Profilage du flux MJPEG et fragmentation réseau

Afin de quantifier précisément les besoins en bande passante du flux MJPEG final, une mesure de métrologie systématique a été menée. Un enregistrement continu d'une minute (60 secondes) a été capturé en conditions réelles en agitant la caméra pour simuler le comportement dynamique d'un vol :

```
rplicam-vid -t 60000 -n -o fpv_test_1min.mjpeg --width 320 --height 240 --framerate 60 --codec mjpeg --denoise off --exposure sport --metering centre --awb daylight --quality 8 --flush
```

L'analyse du fichier binaire obtenu a permis d'établir le profilage mathématique suivant :

1. **Volume de données global** : Le fichier généré pèse exactement 11 534 336 octets (≈ 11.53 Mo).
2. **Volume d'images produit** : À une fréquence d'échantillonnage de 60 Hz sur une durée de 60 secondes, la caméra a produit exactement 3600 images.
3. **Poids moyen d'une frame (Frame Size)** :

$$\text{Taille moyenne} = \frac{11\,534\,336 \text{ octets}}{3600 \text{ images}} \approx \mathbf{3204} \text{ octets par image}$$

4. **Bande passante requise (Throughput)** : Le débit réel consommé par le flux vidéo se calcule en divisant le poids total par le temps d'acquisition :

$$\text{Bande passante} = \frac{11.53 \text{ Mo} \times 8 \text{ bits}}{60 \text{ s}} \approx \mathbf{1.53} \text{ Mbps}$$

Cette résolution et ce framerate ont été trouvés de manière empirique ; c'est le « sweetspot » auquel la vidéo reste fluide avec une qualité acceptable pour l'utilisation. Au-delà de ces paramètres, des pertes sont observées ; on peut donc en déduire que 1.53 Mbps est la limite de charge utile du système.

Ce profilage démontre que l'image MJPEG moyenne (3204 octets) dépasse la taille maximale de la charge utile de nos paquets (MTU applicative fixée à 1400 octets). Une image est donc segmentée en moyenne en **3 paquets UDP** ($3204/1400 \approx 2.28 \rightarrow 3$ paquets).

11.3 Pipelines d'émission MJPEG final (TX)

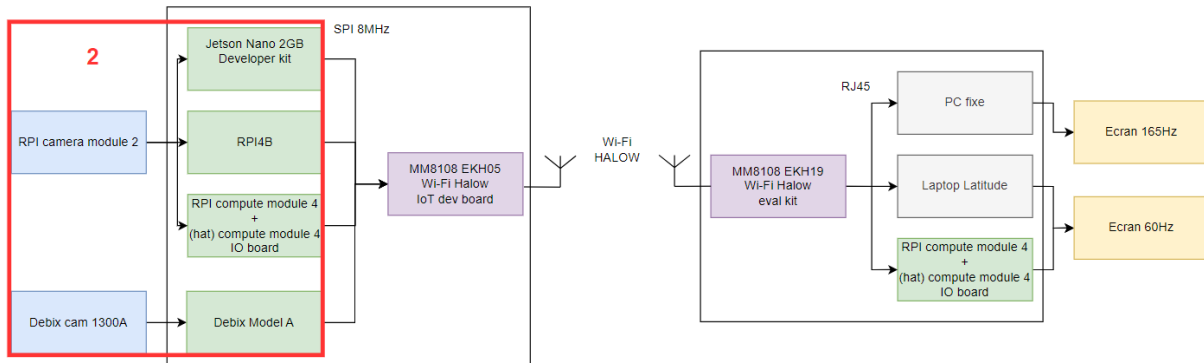


FIGURE 7 – (2) Capture vidéo

Pour implémenter cette stratégie MJPEG ultra-basse latence, le pipeline d'émission s'appuie sur l'outil natif `rpicam-vid` sur la Raspberry Pi (TX), configuré de manière extrêmement agressive pour garantir un temps de calcul déterministe de l'ISP :

```
# Commande d'acquisition et de compression MJPEG finale (Raspberry Pi TX)
rpicam-vid -t 0 -n -o - --width 320 --height 240 --framerate 60 --
  codec mjpeg \
--denoise off --exposure sport --metering centre --awb daylight --
  quality 8 --flush
```

Note : les paramètres optimaux pour le H.264 peuvent être retrouvés dans le rapport intermédiaire.

11.3.1 Précisions sur les paramètres d'envoi

Les arguments passés à la ligne de commande visent à supprimer tout traitement d'image adaptatif ou asynchrone :

- `-flush` : Force l'expulsion immédiate des données JPEG compressées de l'espace noyau vers la sortie standard (`stdout`), court-circuitant les tampons de cache en écriture du système d'exploitation de la Raspberry Pi.
- `-width 320 -height 240 (Downscaling matériel)` : L'ISP du capteur IMX219 effectue un sous-échantillonnage matériel direct sur la matrice de pixels. Cela réduit drastiquement le poids des données à traiter sans solliciter le processeur principal de la carte.
- `-denoise off` : Désactive l'algorithme de réduction du bruit numérique, éliminant ainsi une étape de calcul asynchrone sur l'ISP.
- `-exposure sport et -awb daylight` : Gèle l'exposition automatique sur des temps d'obturation très courts (`sport`) et fixe la balance des blancs. Cela évite les variations du temps de calcul de l'ISP lors des changements brusques de luminosité en vol.

- **-quality 8** : Règle la table de quantification JPEG sur une compression forte. Ce paramètre clé maintient le flux utile moyen autour de 1.5 Mbps, lui permettant de traverser le canal radio de 8 MHz sans saturation.

11.4 Pipeline de réception MJPEG final (RX)

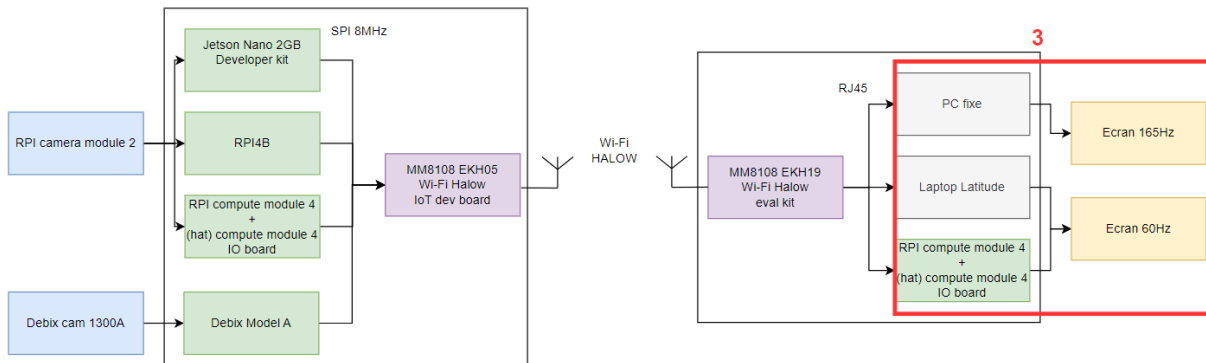


FIGURE 8 – (3) Plateformes de traitement.

À la réception, le flux UDP Unicast arrivant sur le port de la station est décodé et affiché via le pipeline GStreamer épuré de tout mécanisme de synchronisation ou de lissage temporel :

```
# Commande de reception et d'affichage MJPEG finale (PC fixe de
  diagnostic)
gst-launch-1.0 -v udpsrc port=1337 do-timestamp=true ! \
jpegparse ! jpegdec ! queue max-size-buffers=1 leaky=downstream ! \
videoconvert ! autovideosink sync=false
```

11.4.1 Précisions sur les paramètres de réception

- **jpegparse (Réassemblage des fragments)** : Cet élément est indispensable. Le MJPEG générant plusieurs fragments UDP par image, le parseur réassemble les paquets de 1400 octets jusqu'à la détection du marqueur de fin d'image JPEG (0xFF 0xD9), transmettant l'image complète et propre au décodeur **jpegdec** pour éviter tout crash de l'affichage.
- **queue max-size-buffers=1 leaky=downstream** : Limite le stockage en mémoire à une unique image. Si le décodeur subit un retard de traitement, les anciennes images en attente sont immédiatement jetées en aval, forçant l'affichage à recoller instantanément au temps réel sans accumuler de décalage temporel (*buffer bloat*).
- **sync=false** : Désactive la synchronisation du moteur de rendu sur l'horloge globale de GStreamer, affichant l'image immédiatement après sa reconstruction en ignorant les métadonnées temporelles.

12 Interfaçage matériel et transfert SPI

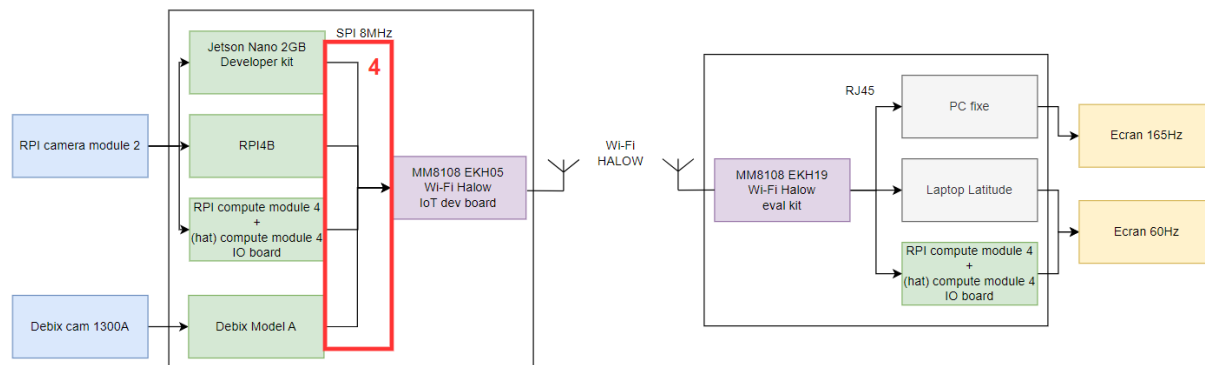


FIGURE 9 – (4) Interfaçage SPI

Le transfert du flux vidéo compressé entre le processeur d'acquisition (Raspberry Pi TX) et le modem radio (STM32) s'effectue via le bus matériel SPI. Cette étape est critique car elle conditionne le rythme d'envoi des données au MCU avant l'émission RF.

12.1 Identification du bus SPI matériel

Une attention particulière a dû être portée à la configuration physique de la liaison. Le SDK Morse Micro ne supportant nativement aucun périphérique standard en dehors de la puce Wi-Fi, le routage exact du bus SPI du microcontrôleur sur le connecteur d'extension n'était pas documenté dans le guide utilisateur de base.

Pour résoudre cette inconnue, une analyse du schéma de conception de la carte **EKH05** [7] a été menée afin d'identifier théoriquement les broches physiques et les pistes reliées aux périphériques internes du processeur. Afin de valider de manière absolue ce schéma avant d'y connecter l'ordinateur d'acquisition, une vérification a été effectuée : une tension de 3,3 V a été appliquée sur les lignes d'entrée repérées, permettant de confirmer au multimètre la correspondance exacte du routage (1).

Broche EKH05	Port STM32	Signal SPI	Description fonctionnelle
D10	PE12	NSS (nCS)	<i>Not Chip Select</i> (Sélection de l'esclave)
D11	PE15	MOSI	<i>Master Out Slave In</i> (Données entrantes)
D12	PE14	MISO	<i>Master In Slave Out</i> (Données sortantes)
D13	PE13	SCK	<i>Serial Clock</i> (Horloge)

TABLE 1 – Routage du bus SPI sur la carte d'évaluation EKH05

12.2 Développement de la Gateway SPI-MJPEG (C)

Le changement de paradigme vers le MJPEG a nécessité la réécriture complète du programme d'interfaçage en C (situé entre l'outil de capture et le bus SPI). En H.264, le flux pouvait être "poussé" aveuglément. En MJPEG, envoyer des morceaux bruts aléatoires à travers le réseau causait des plantages du décodeur GStreamer à la réception, qui ne tolère aucune corruption de structure.

Le programme d'émission (voir code complet en **Annexe A**) agit désormais comme un véritable parseur actif :

1. **Isolation des images** : Il lit le flux continu depuis l'OS en mode non-bloquant et scanne les octets à la volée pour identifier les marqueurs standards du protocole JPEG : 0xFF 0xD8 (Début d'image - SOI) et 0xFF 0xD9 (Fin d'image - EOI).
2. **Slicing à 1400 octets** : Une fois une image complète isolée en RAM, il la fragmente strictement en paquets de 1400 octets. Une capture réseau (`tcpdump`) a permis de valider empiriquement que l'application respectait bien cette taille maximale.
3. **Padding de sécurité** : Si le dernier morceau de l'image fait moins de 1400 octets, le programme complète l'espace vide du tampon SPI avec des zéros (0x00). Le vrai marqueur de fin d'image (0xFF 0xD9) généré par la caméra étant déjà inclus et préservé dans les données utiles, le parseur GStreamer de réception (`jpegparse`) reconstruit la trame et ignore naturellement ce padding neutre sans provoquer d'erreur.

12.3 Contrôle de flux matériel (Handshake SPI)

Le transfert s'effectue sur le bus SPI à une fréquence d'horloge poussée à **8 MHz**. Mathématiquement, l'envoi d'un paquet UDP contenant une charge utile de 1400 octets (soit 11 200 bits) à cette fréquence requiert un temps de transmission physique incompressible. Ce temps théorique se calcule ainsi :

$$T_{SPI} = \frac{1400 \times 8 \text{ bits}}{8\,000\,000 \text{ bits/s}} = 0.0014 \text{ s} = \mathbf{1.40 \text{ ms}}$$

L'architecture UDP étant configurée sans acquittement logiciel, si l'ordinateur enchaîne l'envoi de ces paquets de 1.40 ms trop rapidement, la mémoire RAM du microcontrôleur STM32 déborde (*Buffer Overrun*). Des morceaux d'images sont écrasés avant même d'atteindre le module Wi-Fi, provoquant de sévères artefacts visuels (effets de "bavure" ou de déchirement d'image).

Pour pallier ce problème, un mécanisme de *Handshake* matériel a été implémenté via une broche de signalisation dédiée (GPIO 24 sur la Raspberry Pi vers PD15 sur le STM32) :

```
// Attente active que le STM32 soit PRET (Broche a 1)
while ((* (gpio + 13) & (1 << GPIO_PIN)) == 0) {
    timeout_counter++;
    if (timeout_counter > 2000000) { return 0; }
}
```

Le processus est le suivant : le STM32 abaisse ce signal (niveau BAS) dès qu'il reçoit un paquet de 1400 octets via l'interruption SPI. Il ne le relève (niveau HAUT) que lorsque la couche réseau LwIP a fini de transférer le paquet au module Wi-Fi HaLow. Cette synchronisation matérielle garantit un flux vidéo propre, dictant la cadence de l'émetteur sans aucune perte interne au système.

12.4 Optimisation de l'ordonnancement OS et Overclocking

Afin de garantir un comportement strictement déterministe et d'éliminer la gigue (*jitter*) introduite par l'ordonnanceur multitâche du noyau Linux, deux optimisations système agressives ont été appliquées sur la Raspberry Pi (TX).

12.4.1 Overclocking et verrouillage des fréquences

Par défaut, le noyau Linux de la Raspberry Pi adapte dynamiquement la fréquence du processeur en fonction de la charge et de la température pour économiser l'énergie. Ce comportement dynamique génère des variations de performances et introduit des micro-saccades imprévisibles lors du traitement des interruptions et de l'envoi des blocs de données sur le bus SPI.

Pour figer le système dans son état de performance maximal, la configuration de démarrage (`/boot/firmware/config.txt`) a été modifiée pour overclocker le processeur ARM à une fréquence stable de **2.0 GHz** (`arm_freq=2000`). De plus, le gouverneur CPU de Linux a été configuré sur le mode *Performance* pour interdire toute baisse de fréquence (voir les détails et le fichier de configuration complet en **Annexe B**).

12.4.2 Isolation du cœur CPU

Pour s'assurer que le programme d'émission en C (`gateway_spi.c`) et la capture caméra (`rpicam-vid`) ne soient jamais interrompus par des tâches de fond de l'OS (telles que la gestion des paquets système, des démons ou des entrées/sorties), le cœur numéro 3 du processeur ARM Cortex-A72 a été totalement isolé du planificateur (*scheduler*) de Linux. Cette isolation a été réalisée en ajoutant l'argument de démarrage `isolcpus=3` dans le fichier d'initialisation du noyau (`/boot/firmware/cmdline.txt`). Lors du démarrage, le système d'exploitation s'exécute exclusivement sur les cœurs 0, 1 et 2. Notre exécutable en C est alors lancé en forçant son affinité matérielle sur le cœur 3 à l'aide de l'utilitaire `taskset`, tout en lui attribuant la priorité d'ordonnancement maximale (`nice -n -20`) :

```
# Lancement de la gateway sur le coeur isole 3 avec priorite  
maximale  
sudo taskset -c 3 nice -n -20 ./gateway_spi
```

Cette approche se rapprochant d'un fonctionnement *Bare-Metal* garantit que le cœur 3 est dédié à 100 % au parsing des images et aux transactions SPI matérielles. Cela a permis lorsque l'OS redistribuait les processus en tâche de fond.

13 Itérations matérielles et analyse des performances

Afin d'isoler avec précision les goulots d'étranglement du système et de valider l'architecture finale, une série de mesures de performances a été menée. L'objectif de cette démarche est de quantifier le temps de transit matériel, le temps de transport réseau, puis le temps de traitement de l'image, afin d'établir un bilan "Glass-to-Glass" absolu.

13.1 Profilage théorique du transport matériel

13.1.1 Mesure au cycle d'horloge du traitement MCU (STM32)

Pour profiler le transfert SPI et l'encapsulation LwIP, le registre matériel ARM CoreSight DWT (*Data Watchpoint and Trace*) a été exploité, offrant une résolution de 6.25 nanosecondes.

```
dwt_start = DWT->CYCCNT;
// ... [Encapsulation pbuf_alloc et envoi udp_sendto] ...
dwt_end = DWT->CYCCNT;
uint32_t time_us = (dwt_end - dwt_start) / (SystemCoreClock /
    1000000);
```

L'extraction de cette métrique a révélé qu'avec l'optimisation agressive des files d'attente QoS et la priorité matérielle accordée aux paquets vidéo, le temps de traitement MCU est tombé à environ **0.3 ms par paquet** (contre 0.8 ms initialement), minimisant le temps de transit interne du microcontrôleur.

13.1.2 Bilan de la latence de transport radio pur

Pour isoler la latence de transmission pure, une architecture à "déclencheur commun" a été mise en place (voir Figure 10). Deux Raspberry Pi 4B (TX et RX) ont été reliées par un bouton poussoir commun. Le TX injecte un paquet test via SPI lors de l'appui, et le RX fige son chronomètre haute résolution à la réception.

Les résultats d'un test en rafale (*Burst*) ont donné une moyenne stable de **10 à 15 ms** par paquet. Ce résultat empirique prouve que le transport radio Sub-1 GHz est extrêmement rapide et n'est pas le responsable des lourdes latences observées sur les transmissions vidéo standards.

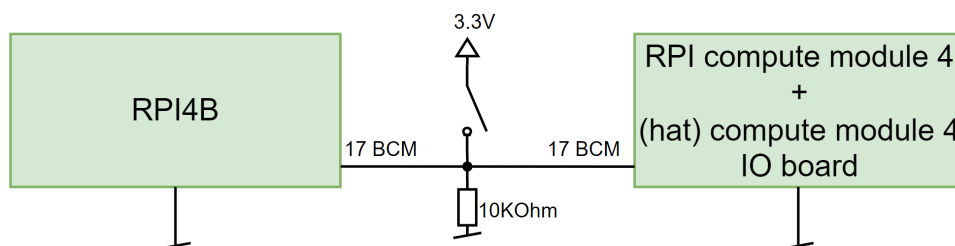


FIGURE 10 – Schéma du système de test de latence matérielle

13.2 Traque de la latence applicative : Le plafond de verre H.264

Lors de la première itération du projet (H.264), la latence "Glass-to-Glass" globale s'élevait entre 100 et 140 ms. Sachant que le temps de vol matériel pur n'excédait pas 15 ms, plus de 100 millisecondes étaient inexplicables.

L'approche initiale pour isoler ce problème via un test local (*loopback* sur la même carte) avait mesuré un délai d'encodage et de décodage local d'environ 50 à 70 ms. Pour obtenir un diagnostic absolu du goulot d'étranglement, le pont radio Wi-Fi a été totalement retiré et remplacé par une connexion réseau filaire directe (câble Ethernet RJ45 direct) reliant l'émetteur et le récepteur.

Le test en Ethernet a confirmé une latence résiduelle incompressible de **100 à 120 ms**. L'élément radio étant absent, la conclusion était claire : la majorité de la latence provenait de la pile logicielle H.264 (buffers du noyau Linux, abstraction V4L2 et surtout la répartition et la rétention temporelle inhérente à la reconstruction d'un codec inter-frame). C'est ce diagnostic clé qui a imposé le pivot complet vers le MJPEG.

13.3 Performances et limites de l'architecture finale

13.3.1 Comparatif avec un système Broadcast alternatif (DVB-T2)

Afin de situer les performances de notre architecture par rapport aux standards de transmission numérique du commerce, une étude comparative a été menée avec la technologie DVB-T2. Ce système de référence exploitait une carte radio logicielle (SDR BladeRF) pour moduler un flux MPEG-TS.

Bien que très robuste en termes de portée, cette approche s'est révélée inexploitable pour le pilotage en temps réel. Les contraintes de multiplexage et les importants tampons mémoires des récepteurs standards induisaient une latence "Glass-to-Glass" mesurée entre **2 et 7 secondes**. Les scripts de configuration et les commandes de ce système de référence sont documentés en **Annexe C**. Ce constat met en lumière la pertinence de l'implémentation directe d'un pont SPI-HaLow dédié pour contourner ces goulots d'étranglement industriels.

13.3.2 Validation matérielle RF et limitations de puissance

Pour s'assurer que le lien radio exploitait au mieux ses capacités, des tests de puissance d'émission ont été menés à l'aide d'un analyseur de spectre en désactivant son mode "low noise" afin de capturer les pics transitoires d'émission lors d'un protocole de *ping flood*. Ces mesures physiques ont mis en évidence un comportement inattendu : alors que le module de réception au sol EKH19 émet bien à sa puissance maximale de **20 dBm**, le module d'émission embarqué (EKH05) s'est avéré bridé à une puissance d'émission de seulement **14 dBm**, restant ainsi bien en deçà de sa limite théorique de 26 dBm. Des discussions techniques sont actuellement en cours avec le constructeur Morse Micro pour comprendre l'origine de cette limitation logicielle ou matérielle. Bien que cette restriction à 14 dBm n'impacte pas la latence à courte portée, elle représente une limite critique pour la portée maximale du drone en vol réel.

13.3.3 Évaluation empirique de la portée et de la pénétration (NLOS)

Afin d'évaluer la robustesse physique de la liaison radio HaLow Sub-1 GHz, des tests de transmission ont été réalisés en conditions réelles dans différents environnements d'obstacles, en gardant à l'esprit la limitation physique transitoire à **14 dBm** de notre module d'émission (TX) :

Configuration	Type d'environnement et d'obstacles	Stabilité du flux et Latence
Extérieur (LOS)	Visibilité directe pure sans obstacles	Flux stable jusqu'à 250 mètres , latence de 40-50 ms.
Intérieur IICT (NLOS)	Traversée de 2 dalles en béton armé (sol/plafond)	Image parfaitement stable, pas de décrochage ni de perte de trames.
Intérieur IICT limite	Au-delà de 2 étages, obstacles métalliques denses cumulés	Apparition sporadique de macro-blocs, puis rupture de la liaison.

TABLE 2 – Portée utile et comportement de la liaison en mode NLOS

Ces essais démontrent de manière éclatante la viabilité de la liaison Sub-1 GHz. Traverser deux dalles complètes en béton au sein des locaux de l'IICT tout en maintenant un débit de 1.53 Mbps est une performance hors de portée pour les systèmes standards 5.8 GHz à puissance égale. L'obtention d'un firmware d'émetteur corrigé permettant d'atteindre les 26 dBm nominaux déblocuera des performances de portée NLOS encore supérieures.

13.3.4 Qualité visuelle en conditions réelles

La forte compression appliquée (Qualité JPEG fixée au niveau 8) et la résolution de 320x240 pixels représentent le « sweetspot » trouvé empiriquement pour maintenir un débit de 1.53 Mbps sans perte de trames. En extérieur, le traitement d'image de l'ISP (lissage automatique du bruit numérique) compense la réduction de résolution spatiale. L'image résultante conserve une netteté suffisante pour distinguer précisément les obstacles en vol.

FIGURE 11 – Captures d'écran du retour vidéo MJPEG (320x240, Qualité 8, 60 FPS) en extérieur

13.3.5 Bilan Glass-to-Glass final

Le système, dans son architecture finale MJPEG et UDP Unicast, a été évalué sur la station de diagnostic en utilisant un écran configuré à un taux de rafraîchissement de **165 Hz**. Ce choix technique est indispensable pour éliminer au maximum la latence liée au temps de rafraîchissement, qui pénalise les mesures sur les affichages classiques à 60 Hz. La méthodologie de métrologie visuelle à base de carrés séquentiels (générant des blocs de couleurs toutes les 5 ms) a permis de collecter une série de mesures empiriques exemptes d'erreur de lecture. Les performances globales du système s'établissent ainsi :

- **Latence moyenne** : Entre **40 ms** et **50 ms**.
- **Latence minimale (Pic bas)** : **30 ms**.
- **Latence maximale (Pic haut)** : **60 ms** (survenant de manière isolée).

L'analyse statistique de ces mesures, représentée graphiquement par la figure ??, confirme la disparition de la dérive temporelle au fil du temps. Le flux vidéo se maintient largement en dessous du seuil de viabilité fixé par le cahier des charges (80 ms), validant ainsi définitivement la faisabilité de l'utilisation du Wi-Fi HaLow pour la transmission vidéo ultra-basse latence.

14 Conclusion

Le pilotage de drones FPV (*First Person View*) en environnement complexe se heurte depuis toujours à l'incapacité des fréquences standards (2.4 GHz et 5.8 GHz) à pénétrer efficacement les obstacles physiques. Ce travail de Bachelor visait à évaluer une solution de rupture en détournant la norme IEEE 802.11ah (Wi-Fi HaLow) pour y faire transiter un flux vidéo en temps réel sur la bande Sub-1 GHz. Au terme de ce projet, l'architecture globale du système a été établie et optimisée jusqu'à ses limites physiques et logicielles pour répondre aux exigences de réactivité du vol en immersion.

Le succès de ce travail repose sur un changement de paradigme logiciel dicté par la réalité des mesures. L'approche initiale en H.264, bien que très économe en bande passante, s'est heurtée à une latence plancher de 100 ms induite par la reconstruction temporelle des codecs inter-frames. Pour briser ce plafond de verre, le système a été entièrement refondu autour d'un codec exclusivement intra-trame (MJPEG). Cette transition a été rendue possible par l'élargissement de la bande passante radio (8 MHz, MCS 1) et le développement d'un programme d'émission en C sur la Raspberry Pi assurant le découpage à la volée des images en paquets de 1400 octets. L'utilisation de l'UDP Unicast couplée à un forçage agressif des paramètres d'accès au canal (QoS MAC) sur le STM32 a finalement permis de supprimer tout délai de rétention réseau.

Ce travail de bachelor a permis d'obtenir des performances excellentes. Les mesures physiques ont prouvé que le transport matériel pur (transfert SPI, traitement MCU, et temps de vol RF) s'effectue en seulement **12.5 millisecondes**. Mesurée sur un affichage à 165 Hz, la latence visuelle globale (*Glass-to-Glass*) s'établit entre **40 et 50 ms en moyenne**, avec des pointes récurrentes à **30 ms**. L'objectif initial de viabilité fixé à 80 ms est largement dépassé, validant définitivement la pertinence du standard Wi-Fi HaLow pour le contrôle de drone en temps réel.

Les notes journalières, le journal de travail et les diagrammes de Gantt associés au développement de ce projet sont documentés et accessibles à l'adresse suivante :

<https://github.com/TschantzN/doc-bachelore-2026>

Nathan Tschantz



15 Note sur l'utilisation de l'intelligence artificielle

Conformément aux directives académiques, l'auteur déclare avoir utilisé des outils d'intelligence artificielle générative (notamment Gemini 3 Flash et Pro) durant la phase de rédaction de ce rapport intermédiaire.

L'utilisation de cet outil a été strictement encadrée et ciblée sur les points suivants :

- **Assistance à la mise en page et structuration LaTeX** : Génération de structures de code (tableaux, indexation, glossaire) et résolution d'erreurs de compilation.
- **Aide à la formulation et synthèse** : Aide à la structuration logique du document (mise en place de l'approche *Bottom-Up*), et reformulation de notes de laboratoire brutes pour améliorer la clarté et la fluidité académique de la rédaction.
- **Optimisation et génération de blocs de code** : Co-rédaction et ajustement de certaines portions de code spécifiques (telles que les configurations de pipelines GStreamer ou des structures en C), ainsi que l'assistance lors de la réalisation de scripts ou de code (C ou Python). Ces éléments ont été générés systématiquement sur la base des propositions algorithmiques et des contraintes logiques dictées par l'auteur.
- **Soutien à la recherche documentaire** : Aide à l'analyse et à la navigation dans le code source du pilote Morse Micro.

Note importante : L'intégralité de l'ingénierie système, de l'architecture globale, de la démarche itérative, de la conception des bancs de test matériels, des protocoles de mesure et des méthodologies de profilage n'a fait l'objet d'aucune génération par IA et provient exclusivement des travaux de l'auteur. De même, l'analyse des journaux d'exécution (logs) du noyau Linux a été réalisée et validée personnellement. Les requêtes (prompts) soumises à l'IA étaient systématiquement fondées sur ces recherches de bas niveau, des données brutes réelles et sur les notes de travail journalières issues des discussions avec Monsieur Mahboob Karimian (Chargé de R&D, Institut IICT, HEIG-VD).

16 Bibliographie

Références

- [1] CaddxFPV. Walksnail moonlight kit - spécifications techniques (latence fpv). <https://www.caddxfpv.com/products/walksnail-moonlight-kit-only>, 2026. Accédé le 15 juillet 2026.
- [2] DeepBlueMbedded. Stm32 delay microsecond & millisecond utility (dwt delay). <https://deepbluembedded.com/stm32-delay-microsecond-millisecond-utility-dwt-delay-timer-delay/>, 2026. Accédé le 15 juillet 2026.
- [3] DJI. Dji o4 air unit pro - spécifications techniques (latence de transmission). <https://store.dji.com/ch/product/dji-o4-air-unit-pro>, 2026. Accédé le 15 juillet 2026.
- [4] GStreamer Project. GStreamer : Open source multimedia framework. <https://gstreamer.freedesktop.org/>, 2026. Accédé le 15 juillet 2026.
- [5] Morse Micro. *Morse Micro OpenWrt 2.8 Web UI User Guide*. Morse Micro, 2024. Documentation technique constructeur.
- [6] Morse Micro. Mm8108-ekh05 user guide (mm8108-ekh05-user-guide.pdf). <https://www.morsemicro.com/>, 2025. Guide utilisateur du module d'évaluation EKH05.
- [7] Morse Micro. *MM8108-EKH05 V2 Design Package (Schematics, Layout and Pinout)*. Morse Micro, 2025. Accédé le 15 juillet 2026.
- [8] Morse Micro. Mm8108-ekh19 user guide (mm6108_mm8108-eval-kit-user-guide-2.8.pdf). <https://www.morsemicro.com/>, 2025. Guide utilisateur du module d'évaluation EKH19.
- [9] Morse Micro. Morse micro iot sdk framework. <https://github.com/MorseMicro/mm-iot-sdk>, 2026. Accédé le 15 juillet 2026.
- [10] Morse Micro. Morse micro openwrt feed and configuration. <https://github.com/MorseMicro/openwrt>, 2026. Accédé le 15 juillet 2026.
- [11] Morse Micro. Morse micro wi-fi halow driver source code. https://github.com/MorseMicro/morse_driver, 2026. Accédé le 15 juillet 2026.
- [12] NVIDIA Corporation. Getting started with jetson nano developer kit. <https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit>, 2026. Accédé le 15 juillet 2026.
- [13] OpenHD Project. Openhd : Introduction. <https://openhdfpv.org/introduction/>, 2026. Accédé le 15 juillet 2026.
- [14] Pi4J Project. Raspberry pi compute module 4 pinout. <https://www.pi4j.com/1.3/pins/rpi-cm4.html>, 2026. Accédé le 15 juillet 2026.
- [15] Raspberry Pi Ltd. Raspberry pi documentation - getting started. <https://www.raspberrypi.com/documentation/computers/getting-started.html>, 2026. Accédé le 15 juillet 2026.

- [16] STMicroelectronics. Stm32u585vit6q datasheet & spécifications. <https://eu.mouser.com/ProductDetail/STMicroelectronics/STM32U585VIT6Q>, 2026. Accédé le 15 juillet 2026.
- [17] Wikipédia. Ieee 802.11ah. https://fr.wikipedia.org/wiki/IEEE_802.11ah, 2026. Accédé le 15 juillet 2026.

A Guide de connexion et d'accès aux équipements

Durant la phase de développement, le système étant distribué sur plusieurs plateformes matérielles dépourvues d'écran (architecture *headless*), des méthodes de connexion spécifiques ont été mises en place pour interagir avec chaque équipement.

A.1 Accès au microcontrôleur d'émission (STM32 / EKH05)

Le module Morse Micro EKH05 intègre un débogueur ST-LINK natif. La connexion physique s'effectue via un simple câble micro-USB relié à la station de développement (PC Windows). Cette liaison fournit deux interfaces virtuelles simultanées :

- **Le flashage et débogage (SWD)** : Géré de manière transparente par STM32CubeIDE via le serveur OpenOCD.
- **La console de logs (UART)** : Accessible via un émulateur de terminal série (tel que PuTTY). Les paramètres de connexion au port COM virtuel sont systématiquement : Baudrate 115200, 8 bits de données, 1 bit d'arrêt, aucune parité.

Du point de vue du réseau HaLow, le STM32 possède l'adresse IP source 192.168.12.2.

A.2 Accès à l'ordinateur d'acquisition (Raspberry Pi 4B)

Pour modifier les scripts de capture vidéo et lancer la gateway sans alourdir le processeur avec une interface graphique, la carte d'acquisition a été configurée pour un accès distant via SSH (*Secure Shell*).

La connexion s'effectue depuis le PC de développement. En cas de déploiement en extérieur (sans réseau local), l'interface Ethernet de la carte est configurée avec une adresse IP statique de repli pour la prise en main rapide.

A.3 Accès au routeur relais de réception (GL.iNet MT3000 / OpenWrt)

Le routeur OpenWrt faisant office de pont réseau (Bridge) est le centre névralgique de la réception au sol. Son adresse IP sur l'interface sans-fil est 192.168.12.1. Son administration s'effectue exclusivement via son interface Ethernet (LAN), qui a été isolée pour éviter tout conflit de routage.

- **Interface Web (LuCI)** : Accessible depuis un navigateur web à l'adresse `http://192.168.13.1`. Elle permet de configurer le mode moniteur ou les paramètres de l'AP sans chiffrement.
- **Accès SSH (Ligne de commande)** : Indispensable pour injecter le module noyau (`morse.ko`) et créer le pont logiciel (`brctl addif br-ahwlan wlan0`). La connexion s'effectue via :
`ssh root@192.168.13.1`

A.4 Accès et adressage des stations de réception (RX)

Pour éviter les conflits de routage avec le système d'exploitation de la station de réception, le réseau FPV est strictement isolé sur le sous-réseau 192.168.12.x, sans passerelle par défaut (aucun accès Internet croisé).

Station de diagnostic (PC Windows) : L'interface réseau Ethernet doit être fixée manuellement sur l'adresse 192.168.12.10 (Masque : 255.255.255.0, Passerelle : 192.168.12.1).

Station cible (RPi Compute Module / RPi 4B) : Lors de la réception "Bare-Metal" sous Linux, l'interface Ethernet nécessite également d'être forcée manuellement sur une adresse statique pour recevoir le flux ponté par le routeur :
IP : 192.168.12.50

A.5 Tableau récapitulatif d'adressage et d'accès

Le tableau 3 résume l'ensemble du plan d'adressage et des interfaces de gestion de l'architecture déployée :

Équipement / Rôle	Interface d'accès	Adresse IP FPV	Remarques
STM32 / EKH05 (Modem Émetteur)	USB (UART / SWD)	192.168.12.2	Reçoit les trames SPI de la Raspberry Pi et les émet en UDP Unicast vers le PC de diagnostic.
Raspberry Pi 4B (Acquisition Vidéo TX)	SSH (Ethernet direct)	<i>Non applicable</i>	Connecté en SPI au modem. Pas d'IP sur le réseau FPV.
GL.iNet MT3000 (Routeur Relais RX)	SSH / Web (LuCI) via LAN	192.168.12.1 (Interface <code>ahwlan</code>)	IP d'administration LAN décalée sur 192.168.13.1 pour éviter les conflits.
PC Windows (Station de diag. RX)	Accès local	192.168.12.10	Passerelle fixée sur 192.168.12.1.
RPi Compute Module 4 (Station cible RX)	SSH (Wi-Fi dev)	192.168.12.50	Fixée manuellement sur l'interface Ethernet (<code>eth0</code>).

TABLE 3 – Récapitulatif des accès et du plan d'adressage (Sous-réseau FPV : 192.168.12.x)

B Modifications du pilote Morse Micro

Afin de forcer l'utilisation d'une modulation robuste (MCS 1) et l'activation du code correcteur d'erreurs (LDPC) en dehors de l'interface standard d'OpenWrt, des modifications directes (Patching) ont été opérées sur le code source C du module noyau (Kernel Driver) de la puce MM8108.

Les modifications ont porté sur les fichiers sources lors de la cross-compilation du noyau Linux pour l'architecture MediaTek Filogic :

- `command.c` : Surcharge de la structure `morse_cmd_req_set_transmission_rate` pour forcer les bits `req.use_ldpc = 1` et injecter le `mcs_index`.
- `usb.c` : Injection de l'appel manuel `morse_cmd_set_fixed_transmission_rate(mors, 1, 1, 0, 0)` directement dans la séquence d'initialisation `morse_usb_probe` au montage de l'interface pour imposer le MCS 1.

L'ensemble de ces modifications a permis de compiler un module `morse.ko` personnalisé, assurant la stabilité de la liaison nécessaire au transport UDP.

C Mesure de latence via bouton partagé

C.1 Le Code TX (Émetteur de test : `latency_tx.c`)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>
#include <sys/mman.h>

#define CHUNK_SIZE 1400
#define SPI_DEVICE "/dev/spidev0.0"

#define GPIO_BTN 17 // Pin physique 11 (Le Bouton)
#define GPIO_HANDSHAKE 24 // Pin physique 18 (Le STM32)

volatile unsigned *gpio;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);

    *(gpio + 2) &= ~(7 << 12); // GPIO 24 en ENTREE
```

```

    *(gpio + 1) &= ~(7 << 21); // GPIO 17 en ENTREE
}

int main() {
    int spi_fd;
    uint32_t speed = 8000000; // 8 MHz
    uint8_t bits = 8;
    uint32_t mode = 0;

    setup_gpiomem();

    spi_fd = open(SPI_DEVICE, O_RDWR);
    if (spi_fd < 0) { perror("SPI_ouverture"); return 1; }

    ioctl(spi_fd, SPI_IOC_WR_MODE, &mode);
    ioctl(spi_fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
    ioctl(spi_fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

    uint8_t buffer[CHUNK_SIZE];
    memset(buffer, 0xAA, CHUNK_SIZE);

    struct spi_ioc_transfer tr = {
        .tx_buf = (uint64_t)(uintptr_t)buffer,
        .rx_buf = 0,
        .len = CHUNK_SIZE,
        .speed_hz = speed,
        .bits_per_word = bits,
        .cs_change = 0,
    };

    int test_step = 0;

    while (1) {
        if (test_step < 10) {
            printf("\n>>> [UNITAIRE %d/10] Appuie sur le bouton  
pour envoyer 1 paquet...\n", test_step + 1);
        } else {
            printf("\n>>> [BURST TEST] Appuie sur le bouton pour  
envoyer la RAFALE de 10 paquets...\n");
        }

        // 1. Attente du BOUTON
        // *(gpio + 13) lit les niveaux de pins d'entrees
        while ( (*(gpio + 13) & (1 << GPIO_BTN)) == 0 );

        if (test_step < 10) {
            // --- ENVOI UNITAIRE ---
            int timeout = 0, stm_ready = 1;

```



```

        while ( (*(gpio + 13) & (1 << GPIO_HANDSHAKE)) == 0 ) {
            if (timeout++ > 5000000) { stm_ready = 0; break; }
        }
        if (stm_ready) {
            ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr);
            printf("_->_Paquet_envoye.\n");
            test_step++;
        } else {
            printf("[ERREUR]_STM32_bloque.\n");
        }
    } else {
        // --- ENVOI EN RAFALE (BURST) ---
        int paquets_ok = 0;
        for (int i = 0; i < 10; i++) {
            int timeout = 0, stm_ready = 1;
            while ( (*(gpio + 13) & (1 << GPIO_HANDSHAKE)) == 0
                ) {
                if (timeout++ > 5000000) { stm_ready = 0; break;
                }; }
            if (!stm_ready) {
                printf("[ERREUR]_STM32_a_plante_au_paquet_%d.\n", i+1);
                break;
            }
            ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr);
            paquets_ok++;
            usleep(100); // 100us pour laisser le STM32 reagir
        }
        if (paquets_ok == 10) {
            printf("_->_Rafale_de_10_paquets_envoyee_!\n");
            test_step = 0; // Reinitialise le cycle de test
        }
    }

    // Anti-rebond du bouton
    usleep(500000);
}

close(spi_fd);
return 0;
}

```

C.2 Le Code RX (Chronomètre de précision : latency_rx.c)

```

#include <stdio.h>
#include <stdlib.h>

```

```
#include <stdint.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <time.h>
#include <pthread.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define GPIO_BTN 17
#define PORT 1337

volatile unsigned *gpio;
volatile uint64_t t_start = 0;
volatile uint64_t t_end = 0;
volatile int packet_count = 0;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);
    *(gpio + 1) &= ~(7 << 21); // GPIO 17 en ENTREE
}

uint64_t get_time_ns() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

// Thread reseau de reception UDP
void* udp_listener(void* arg) {
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in servaddr = {0};
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);
    bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(
        servaddr));

    uint8_t buffer[2000];

    while(1) {
        recvfrom(sockfd, (char *)buffer, sizeof(buffer), 0, NULL,
            NULL);
        t_end = get_time_ns(); // Mise a jour de l'heure d'arrivee
    }
}
```

```

        packet_count++;           // Incrmente le compteur de paquets
        recurs
    }
    return NULL;
}

int main() {
    setup_gpiomem();

    pthread_t thread_id;
    pthread_create(&thread_id, NULL, udp_listener, NULL);

    int test_step = 0;
    double sum_latencies = 0;

    printf(">>>RX_CHRONO_PRET. Initialisation...\n");

    while (1) {
        // Preparation des variables partagees avant l'appui du
        // bouton
        packet_count = 0;

        // 1. Attente de l'appui du bouton physique commun
        while ( (*(gpio + 13) & (1 << GPIO_BTN)) == 0 );

        // 2. Lancement immediat du chronometre de test
        t_start = get_time_ns();

        int target_packets = (test_step < 10) ? 1 : 10;

        // 3. Attente de la reception du ou des paquets reseau
        while (packet_count < target_packets) {
            if (get_time_ns() - t_start > 1500000000ULL) { //
                Timeout de securite de 1.5s
                printf("[ERREUR] Timeout! Seulement %d/%d paquet(s)
                    )recu(s).\n", packet_count, target_packets);
                test_step = 0;
                sum_latencies = 0;
                break;
            }
        }

        // 4. Calculs de latence et Affichage
        if (packet_count == target_packets) {
            double latency_ms = (t_end - t_start) / 1000000.0;

            if (test_step < 10) {
                // Phase 1 : Test unitaire
            }
        }
    }
}

```

```

        printf("Test_%d/10_|_Latence_:_%%.3f_ms\n",
               test_step + 1, latency_ms);
        sum_latencies += latency_ms;

        if (test_step == 9) {
            printf("
            -----\n
            ");
            printf("MOYENNE_UNITAIRE_:_%%.3f_ms\n",
                   sum_latencies / 10.0);
            printf("
            -----\n
            ");
        }
        test_step++;
    } else {
        // Phase 2 : Test en rafale
        printf("=====\\
        n");
        printf("LATENCE_BURST_TOTALE_(10_paquets):_%%.3f_ms
        \n", latency_ms);
        printf("MOYENNE_EN_FLUX_(Latence_/_10)_:_%%.3f_ms
        \n", latency_ms / 10.0);
        printf("=====\\
        n\n");

        // Fin du cycle de test, reinitialisation
        test_step = 0;
        sum_latencies = 0;
    }
}

// Anti-rebond
usleep(500000);
}
return 0;
}

```

D Code de la Gateway active finale (Raspberry Pi TX)

Le code source suivant présente l'implémentation finale en C de notre gateway d'acquisition active. Ce programme lit le flux vidéo MJPEG en continu depuis l'ISP de la caméra, identifie à la volée les délimiteurs JPEG, fragmente les trames en paquets de 1400 octets, et gère le handshake matériel avec le STM32 pour s'adapter dynamiquement au débit utile du lien radio.

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>
#include <sys/mman.h>

#define CHUNK_SIZE 1400
#define SPI_DEVICE "/dev/spidev0.0"
#define GPIO_PIN 24
#define RING_BUFFER_SIZE (1024 * 64)

volatile unsigned *gpio;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);
    *(gpio + 2) &= ~(7 << 12); // GPIO 24 en entree
}

int wait_for_stm32_ready() {
    int timeout_counter = 0;
    // Attente que le STM32 soit PRET (Broche a 1)
    while ((* (gpio + 13) & (1 << GPIO_PIN)) == 0) {
        timeout_counter++;
        if (timeout_counter > 2000000) { return 0; }
    }
    return 1;
}

int main() {
    int spi_fd;
    FILE *cam_pipe;
    int pipe_fd;
    uint32_t speed = 8000000; // 8 MHz
    uint8_t bits = 8;
    uint32_t mode = 0;

    setup_gpiomem();

    spi_fd = open(SPI_DEVICE, O_RDWR);
    if (spi_fd < 0) { perror("SPI_open"); return 1; }
    ioctl(spi_fd, SPI_IOC_WR_MODE, &mode);
```

```

ioctl(spi_fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
ioctl(spi_fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

// Lancement de la capture MJPEG (320x240, 60 FPS, Qualite 8)
const char *cmd = "rpicam-vid -t 0 -n -o - --width 320 --height
  240 --framerate 60"
                "--codec mjpeg --denoise off --exposure sport
                --metering centre"
                "--awb daylight --quality 8 --flush";

printf(">>> Lancement de la capture camera directe...\n");
cam_pipe = popen(cmd, "r");
if (!cam_pipe) {
    perror("Echec du lancement");
    close(spi_fd);
    return 1;
}

pipe_fd = fileno(cam_pipe);
int flags = fcntl(pipe_fd, F_GETFL, 0);
fcntl(pipe_fd, F_SETFL, flags | O_NONBLOCK);

uint8_t spi_buffer[CHUNK_SIZE];
uint8_t *recv_buf = malloc(RING_BUFFER_SIZE);
size_t recv_len = 0;
int drop_counter = 0;

struct spi_ioc_transfer tr = {
    .tx_buf = (uint64_t)(uintptr_t)spi_buffer,
    .rx_buf = 0,
    .len = CHUNK_SIZE,
    .speed_hz = speed,
    .bits_per_word = bits,
    .cs_change = 0,
};

printf(">>> Routage SPI actif : Anti-Flicker & Handshake
materiel\n");

while (1) {
    int n = read(pipe_fd, recv_buf + recv_len, RING_BUFFER_SIZE
        - recv_len);

    if (n < 0) {
        usleep(1000);
        continue;
    }
    if (n == 0) {

```

```

        printf("Flux_camera_interrrompu.\n");
        break;
    }
    recv_len += n;

    while (recv_len > 4) {
        int start_idx = -1;
        int end_idx = -1;

        for (size_t i = 0; i < recv_len - 1; i++) {
            if (recv_buf[i] == 0xFF && recv_buf[i+1] == 0xD8) {
                start_idx = i; break; }
        }
        if (start_idx == -1) { recv_len = 0; break; }

        for (size_t i = start_idx; i < recv_len - 1; i++) {
            if (recv_buf[i] == 0xFF && recv_buf[i+1] == 0xD9) {
                end_idx = i + 1; break; }
        }

        if (start_idx != -1 && end_idx != -1) {
            size_t jpeg_size = end_idx - start_idx + 1;

            uint8_t *frame_buffer = malloc(jpeg_size);
            memcpy(frame_buffer, recv_buf + start_idx,
                jpeg_size);

            size_t remaining = recv_len - (end_idx + 1);
            memmove(recv_buf, recv_buf + end_idx + 1, remaining
                );
            recv_len = remaining;

            size_t bytes_sent = 0;
            int frame_corrupted = 0;

            while (bytes_sent < jpeg_size) {
                size_t to_send = jpeg_size - bytes_sent;
                if (to_send > CHUNK_SIZE) to_send = CHUNK_SIZE;

                memcpy(spi_buffer, frame_buffer + bytes_sent,
                    to_send);

                // Padding neutre avec des zeros si le morceau
                // est plus petit que la MTU SPI (1400 octets)
                if (to_send < CHUNK_SIZE) {
                    memset(spi_buffer + to_send, 0x00,
                        CHUNK_SIZE - to_send);
                }
            }
        }
    }

```

```

        if (wait_for_stm32_ready()) {
            if (ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr)
                < 1) {
                frame_corrupted = 1;
                break;
            }

            // Handshake determiniteste (Acquittement
            // de reception du STM32)
            int ack_timeout = 0;
            while ((* (gpio + 13) & (1 << GPIO_PIN)) !=
                0) {
                ack_timeout++;
                if (ack_timeout > 50000) break; //
                Securite anti-blocage
            }

            drop_counter = 0;
        } else {
            drop_counter++;
            if (drop_counter == 1 || drop_counter % 120
                == 0) {
                printf("[ATTENTION] Timeout STM32! (%d
                    paquets jetes)\n", drop_counter);
            }
            frame_corrupted = 1;
            break;
        }
        bytes_sent += to_send;
    }

    free(frame_buffer);
    if (frame_corrupted) continue;

    } else {
        break;
    }
}

free(recv_buf);
pclose(cam_pipe);
close(spi_fd);
return 0;
}

```


E Firmware MCU du transmetteur (STM32U5 / spi_interface.c)

Le code C ci-dessous présente le module d'intégration principal embarqué sur le micro-contrôleur STM32U5. Il s'occupe d'initialiser le bus SPI esclave à 8 MHz, d'appliquer le profil de QoS agressif (ACI 3 / Voice) sur les files d'attente du modem Wi-Fi HaLow, et de router sans traitement les paquets SPI directement vers la pile réseau LwIP en UDP Unicast.

```
#include <string.h>
#include <endian.h>
#include "mmosal.h"
#include "mmwlan.h"
#include "mmconfig.h"
#include "mmipal.h"
#include "lwip/icmp.h"
#include "lwip/tcpip.h"
#include "lwip/udp.h"
#include "lwip/netif.h"
#include "mm_app_common.h"
#include "stm32u5xx_hal.h"

#define SPI_PAYLOAD_SIZE 1400
#define DEFAULT_UDP_PORT 1337

SPI_HandleTypeDef hspi1;
uint8_t spi_rx_buffer[SPI_PAYLOAD_SIZE];
volatile bool spi_packet_received = false;

// Variables de Profilage DWT
volatile uint32_t dwt_start = 0;
volatile uint32_t dwt_end = 0;
volatile uint32_t mcu_cycles = 0;
extern uint32_t SystemCoreClock; // 160 MHz (160000000 Hz)

static volatile bool is_network_ready = false;

// Callback du statut de liaison radio
static void link_status_callback(const struct mmipal_link_status *
    link_status)
{
    if (link_status->link_state == MMIPAL_LINK_UP) {
        printf("\n>>> CONNECTE A OPENWRT <<<\n");
        is_network_ready = true;
    }
}

// Lancement de la transmission et du pont r seau
static void udp_unicast_tx_start(struct udp_pcb *pcb)
{
```

```

ip_addr_t dest_ip;
IP4_ADDR(ip_2_ip4(&dest_ip), 192, 168, 12, 10); // Station de
    reception final

// Activation du compteur de cycles materiel ARM DWT
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;

printf(">>>PONT_SPI-WIFI_ACTIVE!_En_attente_de_la_Rpi...<<<\n");
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET); // Indique
    "PRET" a la RPi
uint32_t packet_counter = 0;

while (1)
{
    if (spi_packet_received) {
        struct pbuf *p = pbuf_alloc(PBUF_TRANSPORT,
            SPI_PAYLOAD_SIZE, PBUF_REF);
        if (p != NULL) {
            p->payload = (void *)spi_rx_buffer;

            LOCK_TCPIP_CORE();
            udp_sendto(pcb, p, &dest_ip, 1337);
            UNLOCK_TCPIP_CORE();

            pbuf_free(p);
        }

        // Arret du chronometrage DWT
        dwt_end = DWT->CYCCNT;
        mcu_cycles = dwt_end - dwt_start;

        // Affichage diagnostique toutes les 300 trames pour
        // eviter l'asphyxie UART
        if ((packet_counter++ % 300) == 0) {
            uint32_t freq_mhz = SystemCoreClock / 1000000;
            uint32_t time_us = mcu_cycles / freq_mhz;
            printf("[Profilage]_Temps_traitement_MCU:_%lu_us_
                (%lu_cycles)\n", time_us, mcu_cycles);
        }

        spi_packet_received = false;

        // Relance l'attente d'interruption SPI et releve le
        // Handshake
        HAL_SPI_Receive_IT(&hspi1, spi_rx_buffer,

```

```

        SPI_PAYLOAD_SIZE);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET); //
        Handshake pret
    }
}

static struct udp_pcb *init_udp_pcb(void)
{
    struct udp_pcb *pcb = NULL;
    LOCK_TCPIP_CORE();
    pcb = udp_new();
    if (pcb != NULL) {
        udp_bind(pcb, IP_ANY_TYPE, 1337);
    }
    UNLOCK_TCPIP_CORE();
    return pcb;
}

void SPI_Slave_Init(void)
{
    __HAL_RCC_SPI1_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOD_CLK_ENABLE();

    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
        GPIO_PIN_15;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF5_SPI1;
    HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

    hspi1.Instance = SPI1;
    hspi1.Init.Mode = SPI_MODE_SLAVE;
    hspi1.Init.Direction = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
    hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;
    hspi1.Init.NSS = SPI_NSS_HARD_INPUT;
    hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
    hspi1.Init.TIMode = SPI_TIMODE_DISABLED;
    hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLED;

    if (HAL_SPI_Init(&hspi1) != HAL_OK) {
        printf("ERREUR : Echec initialisation SPI1!\n");
    }
}

```

```

    HAL_NVIC_SetPriority(SPI1_IRQn, 5, 0);
    HAL_NVIC_EnableIRQ(SPI1_IRQn);

    // Initialisation de la broche Handshake (PD15)
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    GPIO_InitStructure.Pin = GPIO_PIN_15;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOD, &GPIO_InitStructure);
}

// Interruption de reception complete SPI
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)
{
    if (hspi->Instance == SPI1) {
        dwt_start = DWT->CYCCNT; // Demarre le chrono DWT
        immediatement
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET); //
        Abaisse le Handshake (occupe)
        spi_packet_received = true;
    }
}

void SPI1_IRQHandler(void)
{
    HAL_SPI_IRQHandler(&hspi1);
}

void app_init(void)
{
    printf("\n\n---PONT_EMETTEUR_FPV:RPI->STM32->HALOW---\n\n");

    SPI_Slave_Init();
    HAL_SPI_Receive_IT(&hspi1, spi_rx_buffer, SPI_PAYLOAD_SIZE);

    // Configuration QoS ultra-agressive (ACI 3 / Voice)
    struct mmwlan_qos_queue_params fpv_qos = {
        .aci = 3,
        .aifs = 2,
        .cw_min = 1,
        .cw_max = 1,
        .txop_max_us = 0
    };

    mmwlan_set_default_qos_queue_params(&fpv_qos, 1);
}

```

```
mmwlan_set_power_save_mode(MMWLAN_PS_DISABLED); //
    Desactivation de la veille radio

app_wlan_init();
mmipal_set_link_status_callback(link_status_callback);

printf("Connexion a l'AP OpenWrt en cours...\n");
app_wlan_start();

// Forcage radio (8 MHz de largeur de bande, MCS 1 robuste)
mmwlan_atc_override_rate_control(MMWLAN_MCS_1, MMWLAN_BW_8MHZ,
    MMWLAN_GI_NONE);
printf("Parametrage radio applique : 8 MHz / MCS 1 force.\n");

while (!is_network_ready) {
    mmosal_task_sleep(10);
}

struct udp_pcb *pcb = init_udp_pcb();
if (pcb != NULL) {
    pcb->tos = 0xC0; // Forcage du marquage TOS Voix au niveau
        IP
    udp_unicast_tx_start(pcb);
}
}
```

F Système de transmission alternatif DVB-T2 (SDR / BladeRF)

Dans le cadre d'une étude comparative menée lors des phases intermédiaires du projet, un système de transmission broadcast alternatif basé sur la norme de télévision numérique terrestre **DVB-T2** a été implémenté et évalué. Cette expérimentation visait à confronter l'architecture Wi-Fi HaLow à un protocole industriel standard de diffusion robuste d'un flux MPEG Transport Stream (MPEG-TS) via une radio logicielle (SDR).

F.1 Matériel et architecture logique

L'architecture de test DVB-T2 repose sur les composants suivants :

- **Émetteur (TX)** : Une Raspberry Pi 4B connectée via USB 3.0 à une carte SDR **BladeRF xA9**. La Raspberry Pi assure la capture vidéo et l'encapsulation MPEG-TS en temps réel avant d'injecter le flux binaire directement sur l'interface d'émission de la BladeRF.
- **Récepteur (RX)** : Un PC Windows équipé d'un dongle récepteur DVB-T2 USB standard (DVB Adapter 2).

FIGURE 12 – Architecture logique de la chaîne de transmission comparative DVB-T2

F.2 Configuration de l'émetteur BladeRF (Raspberry Pi)

Pour assurer la transmission, la BladeRF doit être flashée avec son image FPGA correspondante. Les commandes système suivantes permettent de charger l'image en mémoire non volatile, de lister la carte et d'interagir avec l'appareil :

```
# Chargement de la dernière version de l'image FPGA dans la
  mémoire Flash
sudo bladeRF-cli -L hostedxA9-latest.rbf

# Vérification de la détection de la carte sur le bus USB
bladeRF-cli -p

# Entrée en mode interactif pour l'administration de la SDR
sudo bladeRF-cli -i
```

En mode interactif, les paramètres radiofréquences et la transmission du flux sont pilotés par un fichier de configuration dédié nommé `config.conf` :

```
# --- Contenu du fichier config.conf ---
set frequency tx1 634M
set bandwidth tx1 8M
set samplerate tx1 20M
set gain tx1 60

set frequency tx2 634M
set bandwidth tx2 8M
set samplerate tx2 20M
set gain tx2 60

tx config timeout=80000
tx config file=/dev/stdin format=bin channel=1,2
tx start
tx wait
```

Le flux vidéo issu de la caméra est encodé en H.264 par GStreamer (profil *baseline*, compression *ultrafast* sans latence), multiplexé en MPEG-TS et redirigé via un tube anonyme (*pipe*) vers l'entrée standard (`stdin`) de l'utilitaire `bladeRF-cli` s'exécutant sur le cœur de processeur isolé 3 de la Raspberry Pi :

```
# Pipeline d' mission DVB-T2 via redirection de flux standard (
  stdin)
sudo taskset -c 3 nice -n -20 gst-launch-1.0 -q \
  libcamerasrc ! \
  video/x-raw,width=640,height=480,framerate=30/1 ! \
  videoconvert ! queue max-size-buffers=2 leaky=downstream ! \
  x264enc tune=zerolatency speed-preset=ultrafast key-int-max=30
  bitrate=1500 ! \
```

```
video/x-h264,profile=baseline ! \
mpegtsmux alignment=7 ! \
fdsink fd=1 sync=false | \
sudo bladeRF -cli -e "set_samplerate_tx1_20M" \
-e "set_bandwidth_tx1_8M" \
-e "set_frequency_tx1_634M" \
-e "set_gain_tx1_60" \
-e "set_samplerate_tx2_20M" \
-e "set_bandwidth_tx2_8M" \
-e "set_frequency_tx2_634M" \
-e "set_gain_tx2_60" \
-e "tx_config_timeout=80000" \
-e "tx_config_file=/dev/stdin format=bin channel=1,2" \
-e "tx_start" -e "tx_wait"
```

F.3 Configuration du récepteur et décodage (PC Windows)

Deux méthodes d'acquisition ont été expérimentées sur la station de réception au sol afin d'isoler l'origine de la latence de traitement du flux MPEG-TS.

F.3.1 Approche standard : VLC Media Player

La première solution s'appuie sur VLC pour s'accorder sur la fréquence UHF de 634 MHz, extraire le flux réseau démodulé et le renvoyer en UDP local vers GStreamer :

```
# Fermeture des instances VLC actives pour libérer le dongle USB
taskkill /F /IM vlc.exe

# Lancement de l'acquisition DVB-T2 et encapsulation vers l'adresse
  locale loopback
"C:\Program Files (x86)\VideoLAN\VLC\vlc.exe" "dvb-t2://frequency
=634000000:bandwidth=8" :dvb-adapter=2 :dvb-a-modulation=QPSK :
dvb-plp-id=0 :live-caching=50 --sout="#std{access=udp,mux=ts,dst
=127.0.0.1:1234}" --sout-mux-caching=50 --network-caching=50 -I
dummy
```

F.3.2 Approche optimisée : TSDuck

Pour s'affranchir des tampons internes importants de VLC, l'outil d'analyse de flux MPEG **TSDuck** a été configuré pour assurer une réception à très faible latence :

```
# Capture directe du canal physique et redirection brute en UDP via
  TSDuck
tsp.exe -v -I dvb --adapter 2 --delivery-system DVB-T --frequency
634000000 --bandwidth 8000000 --modulation QPSK --transmission-
mode auto --guard-interval auto --high-priority-fec auto --
signal-timeout 10 -O ip 127.0.0.1:1234
```

F.3.3 Pipeline d’affichage GStreamer (Commun aux deux méthodes)

Indépendamment de l’outil d’acquisition (VLC ou TSDuck), le flux de transport MPEG-TS démodulé et réceptionné sur le port local 1234 est extrait, décodé et affiché par GStreamer à l’aide de l’accélération Direct3D 11 :

```
# D codage matériel basse latence du flux MPEG-TS
gst-launch-1.0.exe -v ^
  udpsrc port=1234 caps="video/mpegts" ! ^
  tsdemux ! ^
  queue max-size-buffers=2 max-size-bytes=0 max-size-time=0 leaky=
    downstream ! ^
  h264parse config-interval=-1 ! ^
  avdec_h264 max-threads=1 ! ^
  videoconvert ! ^
  d3d11videosink sync=false
```

F.4 Bilan des mesures et goulots d’étranglement

Bien que la modulation QPSK opérée par la SDR BladeRF offre une excellente immunité face aux perturbations, la latence mesurée s’est avérée totalement rédhibitoire pour le pilotage FPV :

- **Latence globale moyenne** : Comprise entre **2 et 7 secondes**.
- **Origine du goulot d’étranglement** : La structure même du multiplexage MPEG-TS (qui impose l’insertion régulière de tables de signalisation PAT/PMT et de paquets de bourrage pour maintenir un débit constant) combinée aux importants tampons mémoires des pilotes matériels de réception DVB des dongles USB grand public empêche toute réduction substantielle de la latence de transit, validant ainsi la nécessité d’utiliser notre architecture personnalisée Wi-Fi HaLow.

Glossaire

- Couche MAC** : (*Media Access Control*) Sous-couche du modèle réseau OSI chargée de contrôler l'accès physique au support de transmission (l'air, dans notre cas) et de gérer les acquittements.
- DWT** : (*Data Watchpoint and Trace*) Composant matériel intégré aux processeurs ARM Cortex-M permettant, entre autres, de compter les cycles d'horloge du cœur pour un profilage temporel avec une précision de l'ordre de la nanoseconde.
- EKH05 / EKH19** : Cartes d'évaluation (*Evaluation Kits*) développées par Morse Micro intégrant les puces Wi-Fi HaLow MM8108. L'EKH05 dispose d'entrées/sorties matérielles étendues (bus SPI, GPIO) pour l'IoT, tandis que l'EKH19 est conçue pour un interfaçage réseau standard (USB ou routeur).
- FreeRTOS** : Système d'exploitation temps réel (RTOS) open-source pour microcontrôleurs, garantissant l'exécution de tâches avec des délais de traitement déterministes.
- GPIO** : (*General Purpose Input/Output*) Broche physique d'un microcontrôleur pouvant être configurée par logiciel comme entrée ou sortie numérique (utilisée dans ce projet pour le signal de *Handshake*).
- GStreamer** : Framework multimédia open-source reposant sur une architecture de graphes (pipelines). Il permet de chaîner des éléments logiciels pour capturer, encoder, transmettre et décoder de la vidéo avec une latence minimale.
- Jetson Nano** : Nano-ordinateur développé par Nvidia, équipé d'un processeur ARM et d'un processeur graphique (GPU) intégré, particulièrement optimisé pour l'accélération matérielle de l'encodage vidéo (NVENC).
- LDPC** : (*Low-Density Parity-Check*) Algorithme mathématique très performant de correction d'erreurs (FEC), permettant de reconstruire des paquets radio corrompus sans avoir besoin de les retransmettre.
- LwIP** : (*Lightweight IP*) Pile réseau TCP/IP open-source, hautement optimisée pour les systèmes embarqués (microcontrôleurs) disposant de très peu de mémoire vive (RAM).
- MM8108** : Puce SoC (*System on a Chip*) développée par Morse Micro. C'est le composant électronique au cœur de ce projet, chargé de la modulation et de l'émission des ondes Wi-Fi HaLow.
- OpenHD** : Système d'exploitation open-source basé sur Linux, conçu par la communauté FPV pour transmettre de la vidéo numérique longue portée en détournant des puces Wi-Fi standards (2.4 et 5.8 GHz).
- OpenOCD** : (*Open On-Chip Debugger*) Logiciel libre agissant comme un serveur de débogage. Il permet de programmer et d'inspecter la mémoire d'un microcontrôleur depuis un PC via une interface matérielle.
- OpenWrt** : Distribution Linux open-source extrêmement légère, conçue pour remplacer les microprogrammes (*firmwares*) des routeurs réseau, offrant un contrôle bas niveau sur les interfaces de routage.

Raspberry Pi : Gamme de nano-ordinateurs monocartes très populaires tournant sous Linux. La version *Compute Module 4 (CM4)* est une déclinaison miniaturisée sans connecteurs, destinée à l'intégration dans des systèmes industriels ou embarqués.

SPI : (*Serial Peripheral Interface*) Bus de communication matériel synchrone fonctionnant en mode Maître/Esclave. Utilisé ici pour le transfert à très haute vitesse du flux vidéo compressé entre Linux et le microcontrôleur.

ST-LINK : Sonde de débogage matérielle développée par STMicroelectronics, permettant à un ordinateur de flasher du code et de contrôler un microcontrôleur STM32.

STM32 : Famille de microcontrôleurs 32 bits basés sur l'architecture ARM Cortex-M, développée par STMicroelectronics.

STM32CubeIDE : Environnement de développement intégré (IDE) officiel de STMicroelectronics, basé sur Eclipse, utilisé pour écrire, compiler et déboguer le code C du projet embarqué.

Tcpdump : Outil d'analyse réseau en ligne de commande permettant de capturer et d'inspecter les paquets de données (trames) transitant sur une interface réseau matérielle (Wi-Fi ou Ethernet).

Index

Couche MAC, 25

Debix, 21

EKH19, 21

FPV, 20

FreeRTOS, 18, 23

Jetson Nano, 20

LDPC, 18

LwIP, 21, 23

OpenOCD, 24

OpenWrt, 18, 21

Raspberry Pi, 21

SPI, 21

ST-LINK, 24

STM32, 18, 21

STM32CubeIDE, 23

Wi-Fi HaLow

MM8108, 18, 21